

Year Up United Data Analyst Training Academy: Data Visualization with Power BI

Week 10 Student Guide

This courseware is to be distributed to accompany instructor led training.

Table of Contents

<i>Week 10 Learning Objectives</i>	5
<i>Week 10 Learning Outcomes</i>	6
MODULE 1 POWER QUERY FORMULA LANGUAGE (M).....	7
SECTION 1.A INTRODUCING M	8
1.A.1 <i>M Language</i>	9
1.A.2 <i>Programming for non-programmers</i>	10
1.A.3 <i>Let Expressions</i>	11
1.A.4 <i>Expression naming</i>	12
1.A.5 <i>Formatting Expressions</i>	13
1.A.6 <i>Let expression sets</i>	14
1.A.7 <i>Let Expression Flow</i>	15
1.A.8 <i>Let Expression Flow: Out of Order</i>	16
SECTION 1.B M LANGUAGE LEXICON	17
1.B.1 <i>M Expressions and Values</i>	18
1.B.2 <i>Types of Values</i>	19
1.B.3 <i>Primitive Values</i>	20
1.B.4 <i>List Values</i>	21
1.B.5 <i>Fields and Record Values</i>	22
1.B.6 <i>Table Values</i>	23
1.B.7 <i>Function Values</i>	24
1.B.8 <i>Functions are Values</i>	25
1.B.9 <i>M Function Library Reference</i>	26
1.B.10 <i>Expression Evaluation</i>	27
1.B.11 <i>Expression Evaluation Caution</i>	28
1.B.12 <i>Nesting and the Lookup Operator</i>	29
1.B.13 <i>Positional Index Operator</i>	30
1.B.14 <i>Let expressions compared with Lists</i>	31
1.B.15 <i>Lazy verse Eager Evaluation</i>	32
1.B.16 <i>If Expression</i>	33
1.B.17 <i>Each Keyword</i>	34
1.B.18 <i>Comments</i>	35
1.B.19 <i>Operator Order of Evaluation</i>	36
MODULE 2 WORKING WITH DAX	37
SECTION 2.A DAX SYNTAX FUNDAMENTALS	38
2.A.1 <i>Calculations</i>	39
2.A.2 <i>Formulas</i>	40
2.A.3 <i>Formula Evaluation</i>	41
2.A.4 <i>Formula Examples</i>	42
2.A.5 <i>Naming Requirements</i>	43
2.A.6 <i>Fully Qualified Names</i>	44
2.A.7 <i>Naming Examples</i>	45
2.A.8 <i>Operators</i>	46
2.A.9 <i>Operator Precedence</i>	47
2.A.10 <i>Operator Precedence Examples</i>	48

2.A.11	Scalar Data Types	49
2.A.12	Scalar Data Types, continued	50
2.A.13	The Table Data Type	51
2.A.14	Data Type Conversions	52
2.A.15	Implicit Conversions with Operators	53
2.A.16	Function Fundamentals	54
2.A.17	Function Syntax	55
2.A.18	Order of Evaluation	56
2.A.19	Formatting Formulas	57
2.A.20	Using Online Reference Materials	58
2.A.21	Common Functions by Category	59
2.A.22	More Common Functions by Category	60
SECTION 2.B CALCULATED COLUMNS		61
2.B.1	Calculated Columns	62
2.B.2	Calculated Column Examples	63
2.B.3	Row Context	64
2.B.4	RELATED()	65
2.B.5	RELATEDTABLE()	66
2.B.6	USERRELATIONSHIP()	67
SECTION 2.C MEASURES		68
2.C.1	Measures	69
2.C.2	Implicit Measures	70
2.C.3	Filter Context	71
2.C.4	Interactive Filter Context	72
2.C.5	Example Measures	73
2.C.6	Measures verses Calculated Columns	74
MODULE 3 WORKING WITH VISUALIZATIONS		75
SECTION 3.A CUSTOMIZING VISUALIZATIONS AND REPORT INTERACTIONS		76
3.A.24	Formatting Options	77
3.A.25	Report Themes	78
SECTION 3.B REPORT INTERACTIONS		79
3.B.1	Filter Interactions	80
3.B.2	Move Down vs. Expand Hierarchy	81
3.B.3	Drill Down, Navigate Up	82
SECTION 3.C DATA-BASED FORMATTING		83
3.C.1	Customizing Visuals	84
3.C.2	Conditional Formatting	85
3.C.3	Grouping	86
3.C.4	Binning	87
3.C.5	Clustering	88
SECTION 3.D ADDING INTERACTIVE FEATURES		89
3.D.1	Customizing Filter Interactions	90
3.D.2	Customizing Tooltips	91
3.D.3	Drill Mode	92
3.D.4	Drill-through Pages	93
3.D.5	Bookmarks	94
3.D.6	Q&A Visualization - Deprecated	95
3.D.7	Custom Visualizations	96

3.D.8 *Week 10 Wrap-Up*..... 97

Week 10 Learning Objectives

- This week you will learn:
 - ◇ M language fundamentals: the `let/in` structure, named expressions, step ordering, value types, and the dependency-based evaluation model
 - ◇ M operators: the lookup operator `[ ]` for record fields, the positional index operator `{ }` for lists; `if` expressions; the `each` keyword and the `_` placeholder
 - ◇ DAX vs. M / Power Query: similarities, key differences, and guidance on which to use for a given task
 - ◇ DAX formula structure: the `=` sign, naming, fully qualified table and column references, operators, and operator precedence
 - ◇ DAX calculated columns: row context, and the `RELATED()`, `RELATEDTABLE()`, and `USERELATIONSHIP()` functions
 - ◇ DAX measures: implicit vs. explicit measures, filter context
 - ◇ Visualization types and how to match the chart type to the data story
 - ◇ Report usability and accessibility: audience awareness, color contrast, font sizing, and screen-reader considerations
 - ◇ Report interactions: filter context propagation, expand-hierarchy vs. drill-down behavior
 - ◇ Data-based visual formatting: conditional formatting, grouping, binning, and clustering
 - ◇ Advanced interactive features: customized filter interactions, custom tooltips, drill mode, drill-through pages, bookmarks, Q&A visualization, and custom visuals from AppSource

Week 10 Learning Outcomes

- By the end of this week you will be able to:
 - ◇ Read and interpret a Power Query M expression generated by the GUI, identifying the `let/in` structure, the steps, and what each step does
 - ◇ Identify whether a given analytical requirement calls for a calculated column or a measure, and justify the choice
 - ◇ Write a DAX calculated column that references a related table using `RELATED()` to flatten normalized data
 - ◇ Write a DAX measure using `SUM()` or `SUMX()`, and explain how filter context changes its result in different report locations
 - ◇ Build a report page with at least three visualization types, configure drill-down on a hierarchy, and apply conditional formatting

Module 1

Power Query Formula Language (M)

Section 1.A

Introducing M

1.A.1 M Language

- Learning M Language will help you to perform more advanced tasks that the GUI does not handle well
- As you get comfortable with using M Language, you'll find that some work that can be done using the GUI can be done faster by writing M code
 - ◇ The ability to cut and paste M can save a lot of time!

1.A.2 Programming for non-programmers

- M will feel different than other query languages because it was designed with Excel power users in mind, not for programmers
- M's structure allows for expressions to be broken down into individual steps
 - ◇ This makes it easier to conceptualize a long list of tasks that will need to be performed
- M's evaluation engine determines each expression's dependencies and computes the order that expressions should be evaluated in
 - ◇ This makes it easier in the sense that the author does not need to worry as much about when to do things, but rather what needs to be done
 - ◇ This is similar to a spreadsheet

1.A.3 Let Expressions

- A let expression is the first thing that most people will notice when beginning to learn M
- The following query was written by Power Query Editor during the exercise where we prepared data from Northwind
 - ◇ Tip: Use the drop-down for Display Options in the advanced editor to enable word-wrap

Employee
Display Options ▾ ?

```

let
    Source = MySQL.Database("localhost:3306", "northwind", [ReturnSingleDatabase=true]),
    northwind_employees = Source[Schema="northwind",Item="employees"][Data],
    #"Removed Columns" = Table.RemoveColumns(northwind_employees,{"HomePhone", "Address", "Extension", "Photo",
        "Notes", "PhotoPath"}),
    #"Renamed Columns" = Table.RenameColumns(#"Removed Columns",{{"LastName", "Employee Last Name"}, {"FirstName",
        "Employee First Name"}, {"Title", "Employee Title"}, {"TitleOfCourtesy", "Employee Title of Courtesy"},
        {"BirthDate", "Employee Birth Date"}, {"HireDate", "Employee Hire Date"}, {"City", "Employee City"},
        {"Region", "Employee Region"}, {"PostalCode", "Employee Postal Code"}, {"Country", "Employee Country"},
        {"ReportsTo", "EmployeeReportsToEmployeeID"}}}),
    #"Changed Type" = Table.TransformColumnTypes(#"Renamed Columns",{{"Employee Birth Date", type date},
        {"Employee Hire Date", type date}, {"Salary", Currency.Type}}),
    #"Added Custom" = Table.AddColumn(#"Changed Type", "Employee Description", each [Employee Title of Courtesy] &
        " " & [Employee First Name] & " " & [Employee Last Name] & " (" & [Employee Title] & ")"),
    #"Added Custom1" = Table.AddColumn(#"Added Custom", "Employee", each [Employee Last Name] & ", " & [Employee
        First Name])
in
    #"Added Custom1"

```

✓ No syntax errors have been detected.
Done Cancel

1.A.4 Expression naming

- Because named expressions are visible in the designer as steps, they will commonly contain spaces for readability
- When a variable name has spaces in it, place it in quotes and prefix it with a #
- Example

```
let
```

```
    NameOne = "This expression is bound to the name 'NameOne',  
              which does not contain spaces",
```

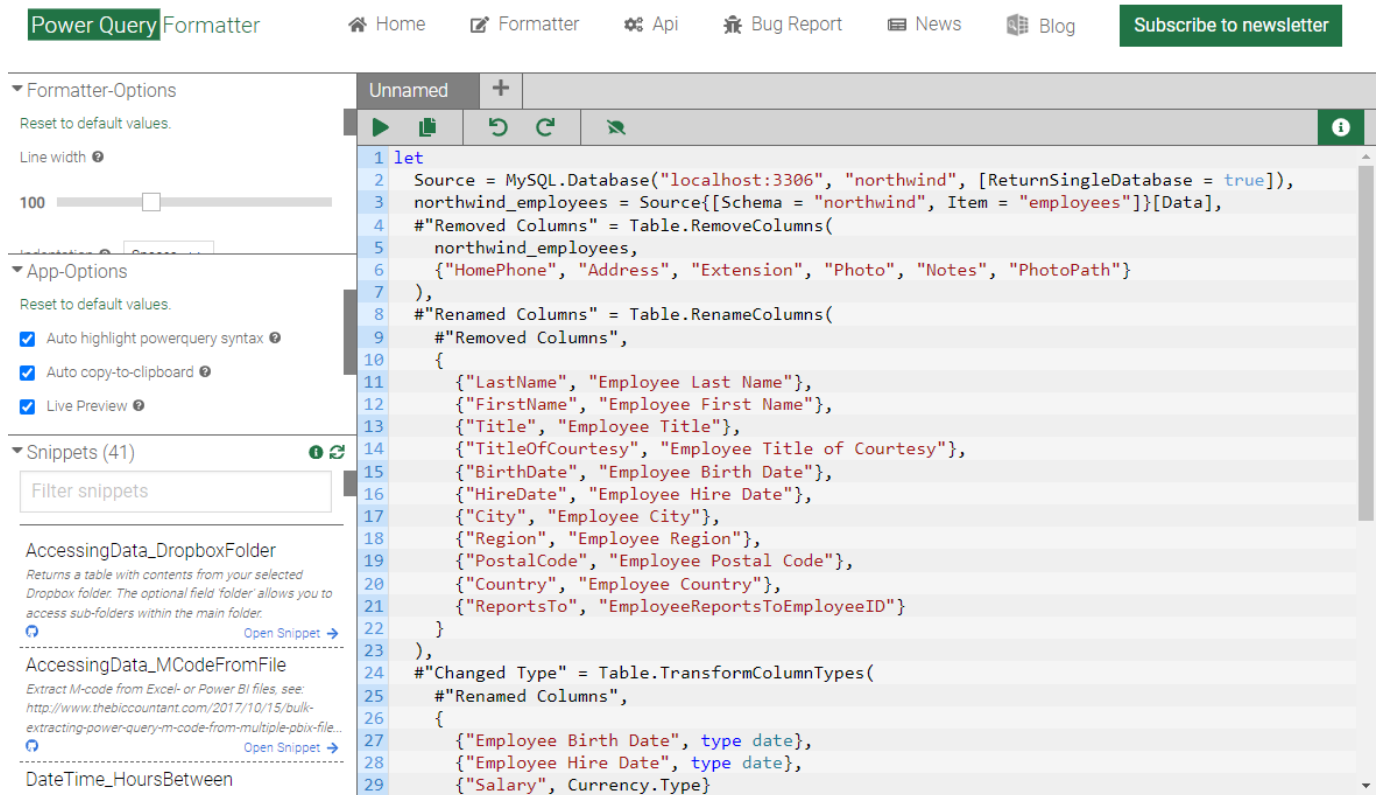
```
    #"Name Two" = "This expression is bound to the name 'Name  
Two, which does contain a space. "
```

```
in
```

```
    #"Name Two"
```

1.A.5 Formatting Expressions

- White space (space, tabs, carriage returns) can be added to your query text, similar to working with SQL
- This can make your query significantly easier to read and write
- Third-party tool <https://www.powerqueryformatter.com/> can be used to format queries to be considerably more readable than what Power Query Editor does on its own



1.A.6 Let expression sets

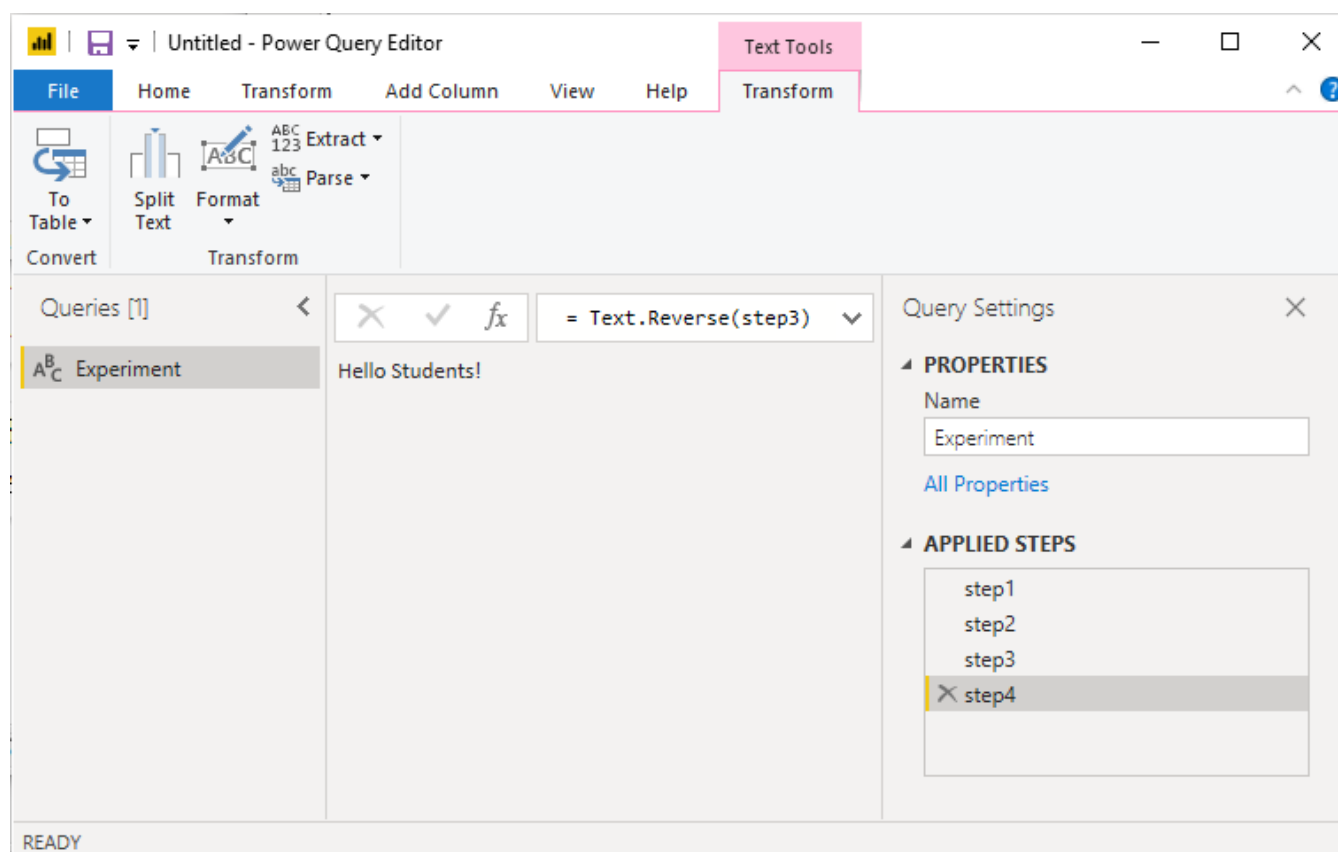
- A **let** expression allows a set of expressions each to be assigned a name
- Each expression can utilize the result of other expressions
 - ◇ Each expression is followed by a comma, except for the last expression before the **in** statement
- A final **in** statement finishes the set, and declares a final result for the expression
 - ◇ Usually this is as simple as the name of an above expression
- Each expression in a **let** statement can reference the result of other expressions

```
1 let
2 Source = MySQL.Database("localhost:3306", "northwind", [ReturnSingleDatabase = true]),
3 northwind_customers = Source{[Schema = "northwind", Item = "customers"]}[Data],
4 #"Removed Columns" = Table.RemoveColumns(northwind_customers, {"Address", "Phone", "Fax"}),
5 #"Renamed Columns" = Table.RenameColumns(
6     #"Removed Columns",
7     {
8         {"CompanyName", "Customer"},
9         {"ContactName", "Customer Contact"},
10        {"ContactTitle", "Customer Contact Title"},
11        {"City", "Customer City"},
12        {"Region", "Customer Region"},
13        {"PostalCode", "Customer Postal Code"},
14        {"Country", "Customer Country"}
15    }
16 )
17 in
18 #"Renamed Columns"
```

1.A.7 Let Expression Flow

- In this example, each step depends only on the steps defined before it:

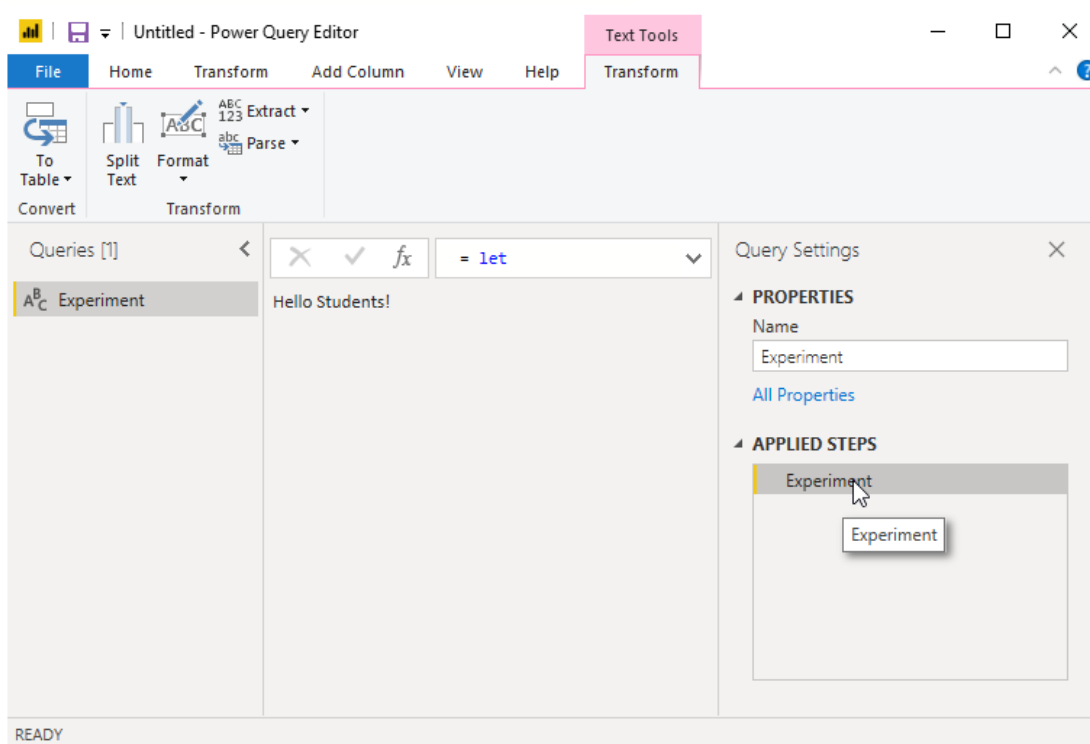
```
let
    step1 = "!stnedutS",
    step2 = "olleH",
    step3 = step1 & " " & step2,
    step4 = Text.Reverse(step3)
in
    step4
```



1.A.8 Let Expression Flow: Out of Order

- In this example we have reversed the order of the steps of the previous example
- Take note that this DOES work!
- However, notice that the applied steps section in the GUI breaks
- Always try to order steps in the let expression so that each step depends only on previous steps

```
let
    step4 = Text.Reverse(step3),
    step3 = step1 & " " & step2,
    step2 = "olleH",
    step1 = "!stnedutS"
in
    step4
```



Section 1.B

M Language Lexicon

1.B.1 M Expressions and Values

- The central construct in M is the expression
- An expression can be **evaluated** (computed) **yielding** (resulting in) a single value
- Many values can be written literally as an expression, but a value is not an expression

◇ Example

- * In this example the expression is 42, and it will evaluate to the value of 42:

```
SomeValue = 42
```

◇ Example

- * In this example the expression is 40 + 2, and it will evaluate to the value of 42:

```
AnotherValue = 40 + 2
```

- An expression is a recipe for evaluation
- Values are the *result* of evaluation

1.B.2 Types of Values

- There are different types of values
 - ◇ Some values are primitive and consist of a single value:
 - * Null
 - * Logical (Boolean)
 - * Number
 - * Time
 - * Date
 - * DateTime
 - * DateTimeZone
 - * Duration
 - * Text
 - * Binary
 - ◇ Others are structured, and consist of a combination of one or more primitives or other structured values:
 - * List
 - * Field
 - * Record
 - * Table
 - * Function
 - * Type

1.B.3 Primitive Values

- A **primitive** value is a single-part value, such as a number, date, text, or null

◇ A null value is used to indicate the absence of any data

Example

```
123           // A number
true          // A logical (or Boolean)
"hello student" // A text
null          // null value
```

- Time-based values are stored as a single value but are expressed through special functions

Example

```
#time(09,15,00)           // a time only
#date(2011,01,26)          // a date only
#datetime(2011,01,06,09,15,00) // a date and a time stored together
#datetimezone(2011,01,06,09,15,00,6,0) // a datetime and offset from UTC
#duration(0,1,30,0)        // a duration of time
```

- Numbers allow for whole numbers and fractions, and can be expressed in scientific notation or hexadecimal

Example

```
3.14 // Fractional number
-1.5 // Fractional number
1.0e3 // Fractional number in scientific notation
123 // Whole number
1e3 // Whole number in scientific notation
0xff // Whole number in hex (255)
```

1.B.4 List Values

- A list value is an ordered sequence of values
 - ◇ M supports infinite lists, but when written as a literal, a list must have a fixed length
 - ◇ Curly brace characters { } denote the beginning and end of a list
 - ◇ Examples
 - * A list containing a number, a logical, and a text:

```
{42, true, "orange"}
```

- * A list of text values:

```
{"red", "orange", "yellow", "green", "blue",  
"indigo", "violet"}
```

1.B.5 Fields and Record Values

- A **field** is a name-value pair where the name is a text value that is unique within the field's record
 - ◇ A field must *belong to* a record
- A **record** is a set of fields
 - ◇ The syntax for record values allows the field names to be written without quotes, a form referred to as **identifiers**
 - ◇ Example
 - * The following example shows a record containing five fields named "A", "B", "C", "D" and "E", which have values 2, 3, 5, 7, and 11:

```
[  
    A = 2,  
    B = 3,  
    C = 5,  
    D = 7,  
    E = 11  
]
```

1.B.6 Table Values

- A **table** is a set of values organized into columns and rows
 - ◇ Columns are identified by names (headers)
 - ◇ There is no literal syntax for creating a table in M, but there are several standard functions that can be used to create tables from lists or records

- ◇ Example

- * This example uses the table function to create a table:

```
#table( {"Product", "Description",  
  {  
    {"Super Soaker", "Pog", "Gameboy"},  
    {  
      "A water gun powered by compressed air",  
      "A collectable cardboard or plastic disk",  
      "An 8-bit handheld game console"  
    }  
  }  
})
```

- * The resulting table would look like this:

Product	Description
Super Soaker	A water gun powered by compressed air
Pog	A collectable cardboard or plastic disk
Gameboy	An 8-bit handheld game console

1.B.7 Function Values

- A function is a value which, when invoked with arguments, produces a new value
- A function in M is written by:
 - ◇ Listing the function parameters in parentheses
 - ◇ Followed by the such-that symbol $\Rightarrow$
 - ◇ Followed by the expression defining the function
 - * The expression typically refers to the parameters by name
 - ◇ Example
 - * The following expression takes the values X, and Y, and returns the average of the two numbers:

$$(x, y) \Rightarrow (x + y) / 2$$

1.B.8 Functions are Values

- A function evaluates to a value, just like a number or text value
- When a function is invoked, a set of values are specified which are logically substituted for the required set of input values within the function body expression

◇ Example

- * This example shows a record with three fields. The first field is a function that is invoked by the expressions of the second and third fields:

```
[
  Add = ( x , y ) => x + y,
  OnePlusOne = Add ( 1, 1 ),
  TwoPlusOne = Add ( 2 , 1)
]
```

- M includes a common set of function definitions available for use in an expression, called the standard library

- ◇ Functions defined in the standard library are available for use without having been explicitly defined by the expression

◇ Example

```
Number.E // Euler's number e (2.7182...)
Text.PositionOf("Hello", "llo") // 2
```

1.B.9 M Function Library Reference

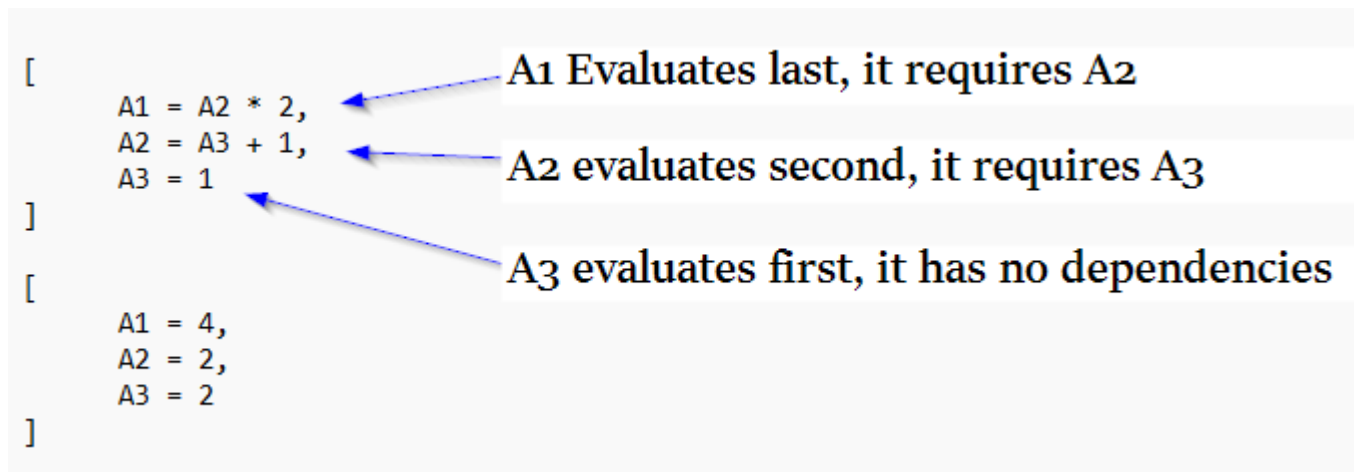
- There are over 700 out of the box functions provided by the M Language Function Library
- Microsoft provides numerous updates each year, and each update brings more functions into availability
- The best reference for this material is Microsoft's online reference found here:

<https://docs.microsoft.com/en-us/powerquery-m/power-query-m-function-reference>

1.B.10 Expression Evaluation

- M Language has an evaluation model that is similar to how a spreadsheet evaluates its expressions
- Parts of an expression can reference other parts of the expression by name
 - ◇ The evaluation process will automatically determine the order in which referenced expressions are calculated
 - * Behind the scenes, it does this by calculating a dependency tree

Example



1.B.11 Expression Evaluation Caution

- Dependency-based expression evaluation is a powerful execution model, but it has a few side effects to be careful with
 - ◇ If an external state is being modified during the evaluation of an expression, then the order that expressions execute would become important to you
 - * If each time a function was executed it returned a different value, and you need those in a predictable order, you would be forced to reverse engineer the dependencies of your expression to understand what is happening
 - ◇ For this reason, try to favor functions that do not change external state
- Whenever possible, try to design deterministic functions
 - ◇ This means that when executed given a set of input values, the output is always the same
 - * A function that removes a column is deterministic
 - * A function that adds two values is deterministic
 - * A function that retrieves data from a database is non-deterministic
 - * A function that subtracts a customer's birthdate from the current time is non-deterministic

1.B.12 Nesting and the Lookup Operator

- Records can be nested within other records
- Use the lookup operator [] to access fields of a record by name
- Example
 - ◇ The following record has a field named Sales containing a record, and a field named Total that accesses the FirstHalf and SecondHalf of the Sales record:

```
[
  Sales = [
    FirstHalf = 1000,
    Second Half = 1100
  ],
  Total = Sales[FirstHalf] + Sales[SecondHalf]
]
```

1.B.13 Positional Index Operator

- Records can be contained within lists
- Use the positional index operator { } to access an item in a list by its numeric index
 - ◇ Values within a list are referred to using a zero based index from the beginning of the list
 - ◇ Example

```
[
    Sales =
        {
            [
                Year = 2017,
                FirstHalf = 2000,
                SecondHalf = 1100,
                Total = FirstHalf + SecondHalf // 3100
            ],
            [
                Year = 2018,
                FirstHalf = 2200,
                SecondHalf = 1300,
                Total = FirstHalf + SecondHalf // 3500
            ]
        },
    TotalSales = Sales{0}[Total] + Sales{1}[Total] // 6600
]
```

1.B.14 Let expressions compared with Lists

- Let expressions might be used to eliminate the need for the positional operator used in the previous example
- Example
 - ◇ Compare this example with the previous one, both return the same result using different techniques:

```
let
    Sales2017 =
        [
            Year = 2017,
            FirstHalf = 2000,
            SecondHalf = 1100,
            Total = FirstHalf + SecondHalf // 3100
        ],
    Sales2018 =
        [
            Year = 2018,
            FirstHalf = 2200,
            SecondHalf = 1300,
            Total = FirstHalf + SecondHalf // 3500
        ]
in
    Sales2017[Total] + Sales2018[Total] // 6600
```

- Although it is not common, it is perfectly acceptable to nest let expressions (a little bit like using subqueries in SQL)

1.B.15 Lazy verse Eager Evaluation

- Lists, Tables, Record Members, and Let expressions are evaluated using **lazy** evaluation
 - ◇ Only as you access a List Item, Record, or Field are those expressions evaluated
 - ◇ This means it is possible that they are never evaluated at all
- All other expressions are evaluated using **eager** evaluation
 - ◇ They are evaluated immediately as encountered during the evaluation process

1.B.16 If Expression

- If can be used to conditionally use one of two expressions, depending on the result of evaluating an initial expression
- Example
 - ◇ In the following example we're testing to see if the customer's age is less than 18, if it is, we'll use the string RED, but if it isn't we'll use the string GREEN

```
if CustomerAge < 18 then
    "RED"
else
    "GREEN"
```

1.B.17 Each Keyword

- The **each** keyword is used in queries when an expression should be performed to create an output for each provided input
 - ◇ Within the function following each, you use the underscore character to represent the current record
 - ◇ If all you need is a fieldname for the current record, the underscore is implied
 - ◇ Example

```
let
    Source = #table( {"col1", "col2"}, { {1, 2}, {10,20} } ),
    AddColumn =
        Table.AddColumn(
            Source,
            "RowTotal",
            each _[col1] + _[col2]
        )
in
    AddColumn
```

1.B.18 Comments

- Comments are used to insert text explaining the query
- Comments are not interpreted as code

<https://docs.microsoft.com/en-us/powerquery-m/comments>

1.B.19 Operator Order of Evaluation

- Similar to algebra, expressions are evaluated in an order based on the operators, not from left to right

Category	Expression	Description
Primary	<i>i</i>	Identifier expression
	@ <i>i</i>	
	(<i>x</i>)	Parenthesized expression
	<i>x</i> [<i>i</i>]	Lookup
	<i>x</i> { <i>y</i> }	Item access
	<i>x</i> (...)	Function invocation
	{ <i>x</i> , <i>y</i> , ...}	List initialization
	[<i>i</i> = <i>x</i> , ...]	Record initialization
	...	Not implemented
Unary	+ <i>x</i>	Identity
	- <i>x</i>	Negation
	not <i>x</i>	Logical negation
Metadata	<i>x</i> meta <i>y</i>	Associate metadata
Multiplicative	<i>x</i> * <i>y</i>	Multiplication
	<i>x</i> / <i>y</i>	Division
Additive	<i>x</i> + <i>y</i>	Addition
	<i>x</i> - <i>y</i>	Subtraction
Relational	<i>x</i> < <i>y</i>	Less than
	<i>x</i> > <i>y</i>	Greater than
	<i>x</i> <= <i>y</i>	Less than or equal
	<i>x</i> >= <i>y</i>	Greater than or equal
Equality	<i>x</i> = <i>y</i>	Equal
	<i>x</i> <> <i>y</i>	Not equal

<https://docs.microsoft.com/en-us/powerquery-m/m-spec-operators>

Module 2

Working with DAX

Section 2.A

DAX Syntax Fundamentals

2.A.1 Calculations

- DAX is used to add values to your existing data model
- There are four types of calculations that can be added to your model with DAX
 - ◇ Calculated Columns
 - * Used to add value at the row level
 - * Can be calculated on a row by row basis as soon as it's added
 - ◇ Measures
 - * Used to add value at a summary level
 - * Requires context to be calculated
 - ◇ Calculated Tables
 - * Can be used to enhance the model with tables derived from other data found in the model
 - ◇ Row Filters
 - * Can be used to provide row-level security
- A calculation is expressed through a formula

2.A.2 Formulas

- A DAX formula always starts with an equal sign (=)
 - ◇ There may be a Name on the left side of the equal sign, to store the result of the formula and which can be used as a field value in a report
- A DAX formula cannot modify values in the model, only add new values
- After the equal sign, you can provide any expression that evaluates to a scalar, or an expression that can be converted to a scalar. These include the following:
 - ◇ Operators, constants, and values provided as part of an expression
 - ◇ References to columns or tables
 - * DAX always uses table and columns as inputs to functions, never an array or arbitrary set of values
 - ◇ The result of a function and its required arguments
 - * Some DAX functions return a table instead of a scalar, and must be wrapped in a function that evaluates the table and returns a scalar

2.A.3 Formula Evaluation

- Formulas need to be evaluated
 - ◇ Evaluation of a formula will produce a result
- Formulas behave differently depending on whether they are used in a calculated column, or in a measure
- Formulas are evaluated with an evaluation context, consisting of:
 - ◇ Row Context
 - * This is relevant only for calculated columns and inside of an iterator
 - * Allows an expression to reference a column value outside of an aggregate
 - ◇ Filter Context
 - * This is most relevant for measures
 - * Both implicit and explicit filters will be applied, which limits the data the formula will see

2.A.4 Formula Examples

- Examples

= 8

The result is the number 8

= "Sales"

The result is the string *Sales*

= 'Sales' [Amount]

When used in a formula defining a value in the Sales table, you will get the value of the [Amount] column for the current *Row Context*

= (0.03 * [Amount])

The result is three percent of the value in the amount column of the current *Row Context* of the current table

= 0.03 * [Amount]

Although this formula can be used to calculate a percentage, the results is not shown as a percentage unless you apply formatting in the table

=PI ()

The value of the constant of π

2.A.5 Naming Requirements

- Each table must have a unique name (just like in SQL databases)
- Within a table, each column must have a unique name
- Object names are case-insensitive
 - ◇ SALES is the same as Sales
- Each column or measure you add to a data model must belong to a specific table

2.A.6 Fully Qualified Names

- When you use a table or column as an input to a function you must **qualify** the name
- To accommodate spaces, reserved keywords, or disallowed characters, table names are specified inside single quote marks: 'European Sales', 'Номер телефона'
 - ◇ This is not required for simple one-word names
- Column names are specified inside square brackets: [Amount]
 - ◇ This is always required
- A fully qualified column name includes both the table and the column: 'Online Sales' [Sales Amount]
- A fully qualified name is required when you reference a column in the following contexts
 - ◇ As an argument to the function VALUES()
 - ◇ As an argument to the functions ALL() or ALLEXCEPT()
 - ◇ As an argument to the function RELATEDTABLE()
 - ◇ As an argument to any time intelligence function
 - ◇ In a filter argument for the functions CALCULATE() or CALCULATETABLE()

2.A.7 Naming Examples

Object Type	Example
Table Name	Sales
Table Name	'European Sales'
Fully Qualified Column Name	Sales[Amount]
Fully Qualified Measure Name	Sales[YTD Profit]
Unqualified column name	[Amount]
Fully qualified column name	'Canada Sales'[Qty]

2.A.8 Operators

- There are different categories of operators, used in different scenarios
 - ◇ Arithmetic Operators: +, -, *, /, ^
 - ◇ Comparison Operators: =, ==, >, <, >=, <=, <>
 - ◇ Text Concatenation Operator: &
 - ◇ Logical Operators: &&, ||, IN
- Many operators are often combined within a single formula
 - ◇ See: <https://docs.microsoft.com/en-us/dax/dax-operator-reference>

2.A.9 Operator Precedence

- Most formulas contain multiple calculations
- The order in which calculations are performed can affect the return value
- Operator evaluation takes place in order from left to right
- The order is:
 1. Anything in parentheses ()
 2. Exponent operator ^
 3. Multiplication and Division * , /
 4. NOT Operator !
 5. Addition and Subtraction + , -
 6. Concatenation &
 7. Comparison = , == , < , > , <= , >= , <>
- As with other programming languages like SQL and Python, make use of parenthesis to control the order of evaluation

2.A.10 Operator Precedence Examples

- Examples:

In this example we're computing just a single operator in each step to illustrate the order of operations

```
= 3 + 3 / 1 * 4 - 2
= 3 + 3 * 4 - 2
= 3 + 12 - 2
= 15 - 2
= 13
```

In this example we've added parenthesis to the expression above which changes the order of evaluation and results in a different answer

```
= (3 + 3) / 1 * (4 - 2)
= 6 / 1 * (4 - 2)
= 6 / 1 * 2
= 6 * 2
= 12
```

In this example, notice that the minus '-' sign is an operator, and participates in order of operations! This kind of behavior may not be expected as it differs from algebra

```
= -2^2
// this results to -4
```

```
= (-2)^2
// this results to 4
```


2.A.11 Scalar Data Types

- Each value has a type, and the type depends on how the source column was defined
- A data type restricts the data that can be stored in a table and helps the operators to know how to interpret the value
- When writing a calculated column or measure, the formula must return a scalar value (i.e. a single value)
- DAX works with the following scalar data types
 - ◇ Integer
 - * Numbers that have no decimal places
 - * Can be positive or negative
 - * Whole numbers between -9,223,372,036,854,775,808 (-2^{63}) and 9,223,372,036,854,775,807 ($2^{63}-1$)
 - ◇ Real Number
 - * Numbers that can have decimal places
 - * Negative values from $-1.79E +308$ through $-2.23E -308$; zero; and positive values from $2.23E -308$ through $1.79E + 308$
 - * However the number of significant digits is limited to 15 decimal digits
 - ◇ Boolean (True or False)
 - ◇ String (alphanumeric unicode text)

2.A.12 Scalar Data Types, continued

◇ Date/Time

- * Dates and Times in an accepted date-time representation
- * Valid dates are all dates after December 30th 1899
- * Stored as a floating point value
- * Integer part is number of days after December 30th 1899
- * Decimal part one second is equal to $1 / (24 * 60 * 60)$
- * You can Add 1 to a date to increase the day by 1
- * You can Subtract 1 from a date to decrease the day by 1

◇ Currency

- * Values between -922,337,203,685,477.5808 to 922,337,203,685,477.5807 with four decimal digits of fixed precision.

◇ Blank

- * A blank is a data type in DAX that represents and replaces SQL nulls. You can create a blank by using the BLANK function, and test for blanks by using the logical function, ISBLANK.

2.A.13 The Table Data Type

- In addition to scalar data types, DAX uses a table data type
 - ◇ Many functions require a reference to a table
 - ◇ Other functions return a table that can then be used as input to other functions.
 - ◇ In some functions that require a table as input, you can specify an expression that evaluates to a table
 - ◇ For some functions a reference to a base table is required, an evaluated table will not work
- When writing a calculated table with DAX, the formula must return a table value

2.A.14 Data Type Conversions

- Each DAX function has specific requirements as to the types of data that are used as inputs and outputs.
 - ◇ A function may require an integer for one argument and a date for another
 - ◇ Other functions require text or tables
- DAX will attempt to automatically convert data into the correct type
 - ◇ You can type a date as a string and DAX will parse the string and attempt to cast it as one of the windows date and time formats
 - * The following expression returns a date: = "7/4/2019"
 - * This following expression returns the same date: "July 4th 2019"
 - ◇ TRUE is implicitly converted to the number 1
 - * The following expression returns 3: = TRUE + 2
 - ◇ If you add values in two columns where one is a number represented as text "17" and the other as a number 17, DAX implicitly converts the string to a number
 - ◇ The following expression returns 24: = "12" + 12
- This is called implicit conversion
- If the implicit conversion fails, DAX will return an error

2.A.15 Implicit Conversions with Operators

- Each operator has its own set of rules regarding how it deals with operating on data of different types
- In most cases these rules are intuitive
 - ◇ For example a `REAL * CURRENCY` becomes a `CURRENCY`
 - ◇ This makes sense, as a 2.5 of a \$20 item might cost \$50
- In other cases the rules may be unexpected
 - ◇ For example a `REAL + CURRENCY` becomes a `REAL`
 - * But then, when would you add a currency to a number that isn't a currency?
- Some situations are very unusual
 - ◇ For example a `DateTime / DateTime` becomes a `REAL`
 - * But you would usually use date functions to do this kind of work
- It's best to check the Microsoft documentation where you have questions, as it provides a full set of tables that define the rules for each operator

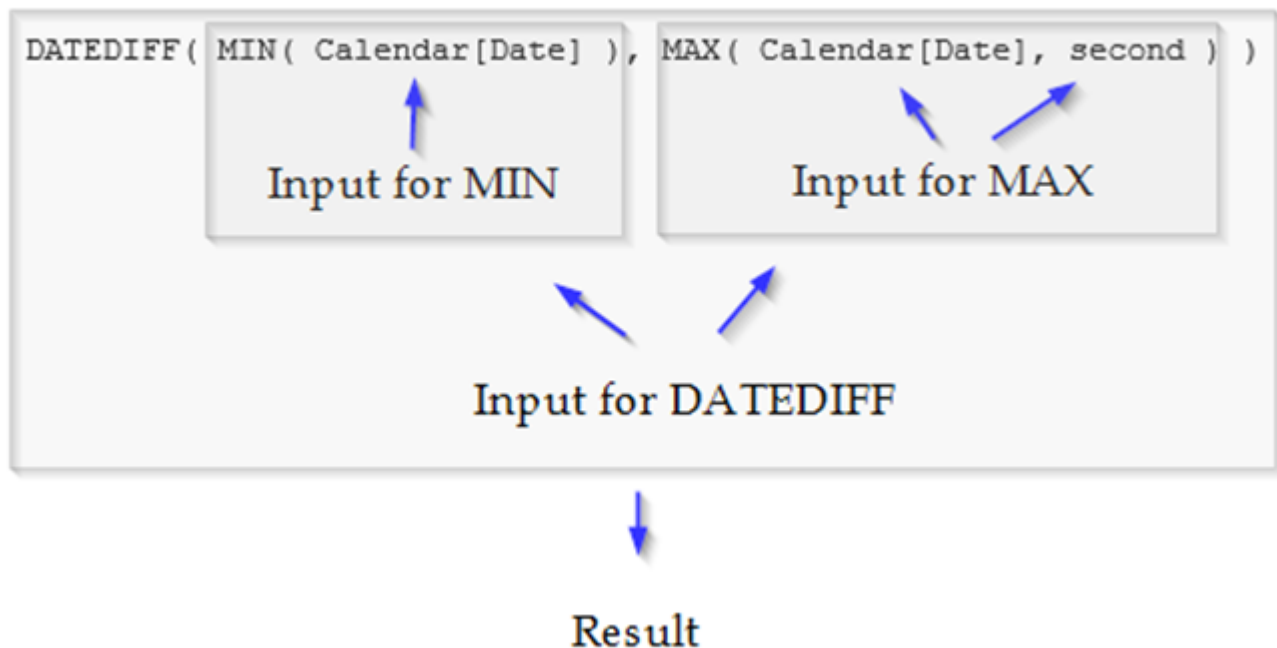
2.A.16 Function Fundamentals

- DAX is a functional language, the execution flows with function calls
- Each Function consists of
 - ◇ Its name
 - ◇ A list of input parameters
 - ◇ A return type (implicit)
- We'll be covering many functions as the course continues
- Microsoft's reference can be found here:
<https://docs.microsoft.com/en-us/dax/dax-function-reference>
- There are over 250 functions available in numerous categories

2.A.17 Function Syntax

- Your first step to learning a new function should be to reference the Microsoft documentation provided online for that function
- When calling a function, you will pass arguments into the input parameters, inside parenthesis
- Example

```
DATEDIFF (MIN (Calendar[Date] ), MAX (Calendar[Date] , second ) )
```



2.A.18 Order of Evaluation

- Functions are evaluated from the inside out

- Example

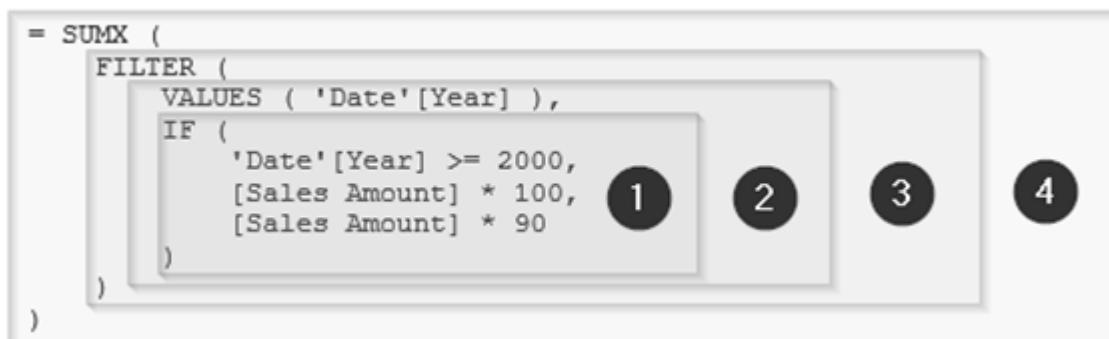
This formula

```
= SUMX(FILTER(VALUES('Date'[Year]),IF('Date'[Year] >= 2000,[Sales Amount] * 100, [Sales Amount] * 90)))
```

- Can be expanded into a more readable format:

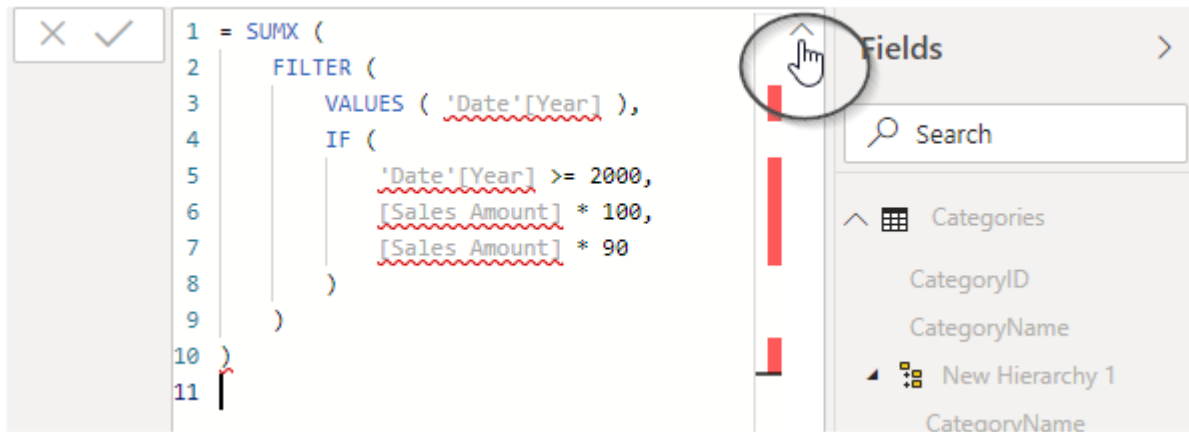
```
= SUMX (
    FILTER (
        VALUES ( 'Date'[Year] ),
        IF (
            'Date'[Year] >= 2000,
            [Sales Amount] * 100,
            [Sales Amount] * 90
        )
    )
)
```

- This gets executed from the inside out:



2.A.19 Formatting Formulas

- Power BI allows you to expand the formula bar to type multiple lines at once



- You will find yourself thinking about writing formulas from the outside in or sometimes the inside out, instead of from left to right, or top to bottom

2.A.20 Using Online Reference Materials

- A quick web search to bring you to the official Microsoft reference for a specific function will save you time
- With over 250 functions and new ones added multiple times a year, it wouldn't hurt to spend a little time browsing through the Microsoft directory
- <https://docs.microsoft.com/en-us/dax/dax-function-reference>

2.A.21 Common Functions by Category

- Aggregation Functions

- ◇ SUM()
- ◇ AVERAGE()
- ◇ MIN()
- ◇ MAX()
- ◇ SUMX() ... *and other X functions*

- Counting Functions

- ◇ COUNT()
- ◇ COUNTA()
- ◇ COUNTBLANK()
- ◇ COUNTROWS()
- ◇ DISTINCTCOUNT()

- Logical Functions

- ◇ AND(), OR(), NOT()
- ◇ IF()
- ◇ IFERROR()
- ◇ SWITCH()

2.A.22 More Common Functions by Category

- Information Functions

- ◇ ISBLANK()
- ◇ ISNUMBER()
- ◇ ISTEXT()
- ◇ ISNONTEXT()
- ◇ ISERROR()

- Text Functions

- * CONCATENATE()
- * REPLACE()
- * SEARCH()
- * UPPER()
- * FIXED()

- Date Functions

- * DATE()
- * HOUR()
- * NOW()
- * EOMONTH()
- * WEEKDAY()
- * DATEDIFF()

Section 2.B

Calculated Columns

2.B.1 Calculated Columns

- Calculated columns can be used to add a new field to a table within your model
 - ◇ A Calculated Column is computed for each row of data within the table it belongs
 - ◇ A Calculated Column's value can be used in other calculations as if it was a regular column
- As soon as you create a calculated column in the user interface, values are immediately calculated for each row and displayed within the table
 - ◇ Data is recalculated when you process (refresh) your data
 - ◇ Data is lost when the model is unloaded from memory, and then recalculated when the data is re-loaded
 - * This happens when you close and open Power BI Desktop as the data does not remain in memory after the program closes
- Calculated columns are not supported for models using a DirectQuery realtime data connection
 - ◇ This is because Power BI has no way of knowing when external records change, which may cause column values to need to be recalculated
 - ◇ Only data in the current row can be used in an expression
 - ◇ DirectQuery provides the advantage of real-time data, but this is one of many disadvantages

2.B.2 Calculated Column Examples

- Combine First Name and Last Name columns into a Full Name column

```
[Full Name] = Customer[Last Name] & ", " & Customer[First Name]
```

- Add a new field that identifies with a boolean if an order shipped late

```
[Shipped Late] = [ShipDate] > [RequiredDate]
```

- Combine the year and quarter into a single field for display purposes

```
[Quarter Name] = [Year] & " Q" & [Quarter]
```

- Calculate a line item net sale amount

```
[Net Sale] = [Price] * [Quantity] * ( 1 - [DiscountPercent] )
```

2.B.3 Row Context

- When a calculated column is being evaluated, it is evaluated once for each row, with the context of that row
- This is called **row context**
- To consider values outside of the context of the row that is being processed, you need to use a special function to walk the relationships of the model
 - ◇ RELATED()
 - ◇ RELATEDTABLE()
 - ◇ USERELATIONSHIP()

2.B.4 RELATED()

- RELATED() is used to navigate a predefined active relationship to grab columns from another table
- RELATED() only works to follow a relationship where only a single row would be returned
 - ◇ This means the row context table would need to be the many side of a one-to-many relationship
 - ◇ RELATED also works with one-to-one relationships
 - ◇ Example

With Calculated Columns, it is common to use RELATED() to flatten normalized data. For example, say our database has three tables: Product, ProductSubcategory, and ProductCategory that we want to flatten into Product. If all we are interested in are the Category and Subcategory names, we can add the following calculated columns to Products:

```
[Subcategory Name] =  
    RELATED (ProductSubcategory[SubcategoryName])
```

```
[Category Name] =  
    RELATED (ProductCategory[CategoryName])
```

2.B.5 RELATEDTABLE()

- RELATEDTABLE() is used to navigate a predefined active relationship to grab multiple rows from a related table
 - ◇ It returns a table value
- The result of RELATEDTABLE() needs to be turned into a scalar value before it can be useful to a Calculated Column, which must return a scalar value
- Iterator functions (e.g. SUMX, COUNTX) can be used to turn a table value into a scalar result
 - ◇ Example

In this example, the table value result of RELATEDTABLE() is passed to the SUMX() Function, which is an iterator. The SUMX() function will add up all of the values in the LineTotal column of the SalesOrderDetail Table, but only for rows that relate to the current Row Context

```
TotalSales = SUMX(  
    RELATEDTABLE(SalesOrderDetail),  
    SalesOrderDetail[LineTotal]  
)
```

SUMX(), MINX(), MAXX(), COUNTX() are all iterators and might be used with the result of RELATEDTABLE()

2.B.6 USERELATIONSHIP()

- USERELATIONSHIP() is used inside CALCULATE() to temporarily activate an inactive relationship for the scope of that calculation
 - ◇ Use it when you need a measure to follow a different relationship than the active default (for example, to aggregate sales by ship date instead of order date)
- This is handy because xVelocity data models only allow a single active relationship between two tables
 - ◇ If more than one relationship is added, all but one will be disabled to solve any issues with ambiguity
 - ◇ This frequently comes up when working with Date tables
 - * For example an order might have the following dates:
 - Order Date
 - Arrive By Date
 - Ship Date
- While this solution is fast and easy, another solution might be to utilize role playing tables (covered later)

Section 2.C

Measures

2.C.1 Measures

- Measures are the second of four types of calculations that can be added to a model with DAX
- Measures are possibly the most important feature of DAX!
 - ◇ This is because they are providing summaries which can concisely tell someone what all the detail below it means
 - ◇ This kind of calculation cannot be done in Power Query or SQL without severe performance implications
- Measures are added at the table level *but* they do not provide one value per row
- Measures are computed in real-time when a report queries the data model
 - ◇ They are not computed when you create them but rather only when they are called in a report

2.C.2 Implicit Measures

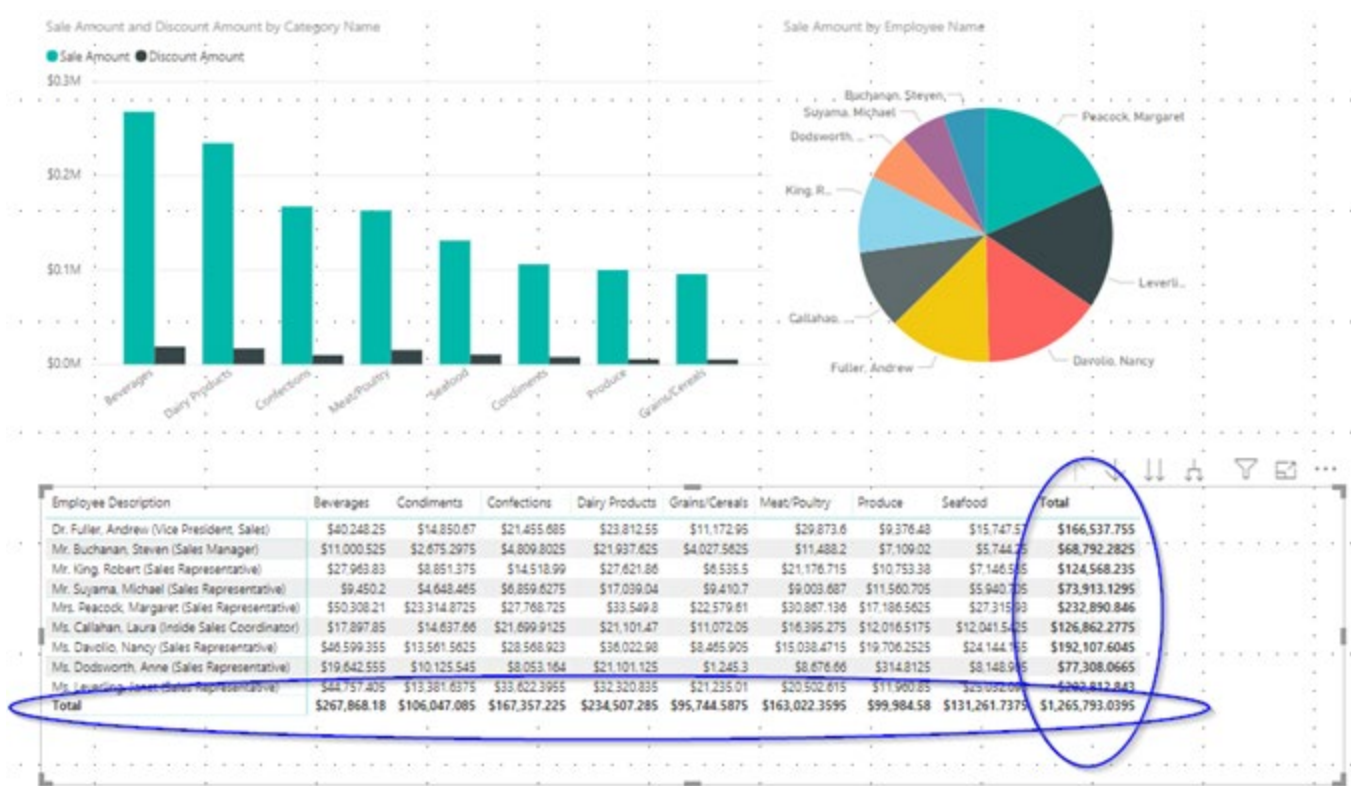
- Any time you add a column to a visual and what you see is an aggregate of many values instead of a single value from a single row, you are using an **implicit measure**
- Notice below that Sale Amount has been added to a visualization. Implicitly, this is actually the formula: `Sum (Sales [Sale Amount] )`

The screenshot displays the Microsoft Power BI interface with three main panes: Filters, Visualizations, and Fields.

- Filters Pane:** Contains filters for 'Category Name is (All)', 'Employee Description is (All)', 'ProductName is (All)', and 'Sale Amount is (All)'. There are also sections for 'Filters on this page' and 'Filters on all pages'.
- Visualizations Pane:** Shows a list of visualization types. Below, the 'Rows' section contains 'Employee Description'. The 'Columns' section contains 'Category Name Hierarchy', 'Category Name', and 'ProductName'. The 'Values' section contains 'Sale Amount', which is circled in blue. Below the Values section are 'Drillthrough', 'Cross-report', and 'Keep all filters' options.
- Fields Pane:** Contains a search bar and a list of data sources. The 'Employees' source is selected. Under 'Products', 'Category Name Hierarchy' is selected. Under 'Sales', 'Sale Amount' is selected and circled in blue.

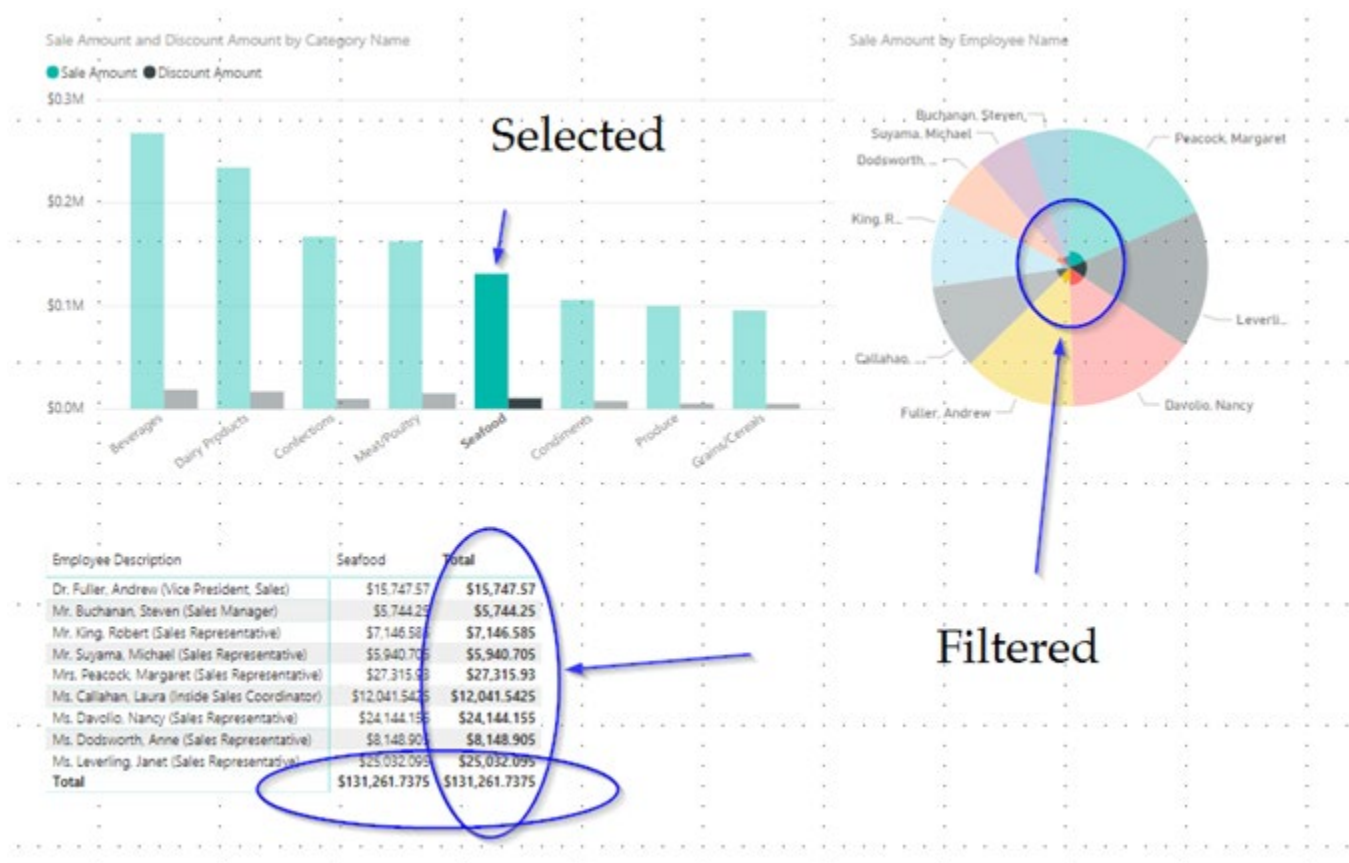
2.C.3 Filter Context

- How data gets summarized is affected by the **filter context**
 - ◇ In the image below you see many numbers, but every single number is the result of the same expression:
 - * `SUM(Sales[Sales Amount])`
 - ◇ The result of the expression is different at each location on the grid because the filter context is different, so a different subset of rows are available to the calculation



2.C.4 Interactive Filter Context

- When a user interacts with report items, selecting an area changes the filter context
- All other report items will limit themselves to display data based on this new filter context



2.C.5 Example Measures

- This formula creates a "Total quantity sold" measure:

```
[Total quantity sold] = SUM(Sales[Quantity])
```

- ◇ Although such a simple measure can be created implicitly by selecting Sales[Quantity] in a report, defining it like this gives you the opportunity to establish a better name for the calculation

- Compute the actual sales values for the given filter context:

```
[Actual sales values] =  
    SUMX(Sales, [Price] * [Quantity])
```

- Compute the list price values for the given filter context:

```
[List price values] =  
    SUMX(Sales, RELATED('Product'[ListPrice]) * [Quantity])
```

- Establish the percentage the customer paid of the list price:

```
[Pct of full value] =  
    DIVIDE(  
        SUMX(  
            Sales, [Price] * [Quantity]  
        ),  
        SUMX(  
            Sales,  
            RELATED('Product'[ListPrice]) * [Quantity]  
        )  
    )
```

2.C.6 Measures verses Calculated Columns

- Use a calculated column when
 - ◇ You will later want to filter on the resulting value
 - ◇ Need to use the value to support another calculated column
- Use a measure when
 - ◇ You want to use percentages in a report
 - ◇ You want to use ratios in a report
 - ◇ You need complex aggregations
 - ◇ You are describing how you'll present a value that will look different in different filter contexts

Module 3

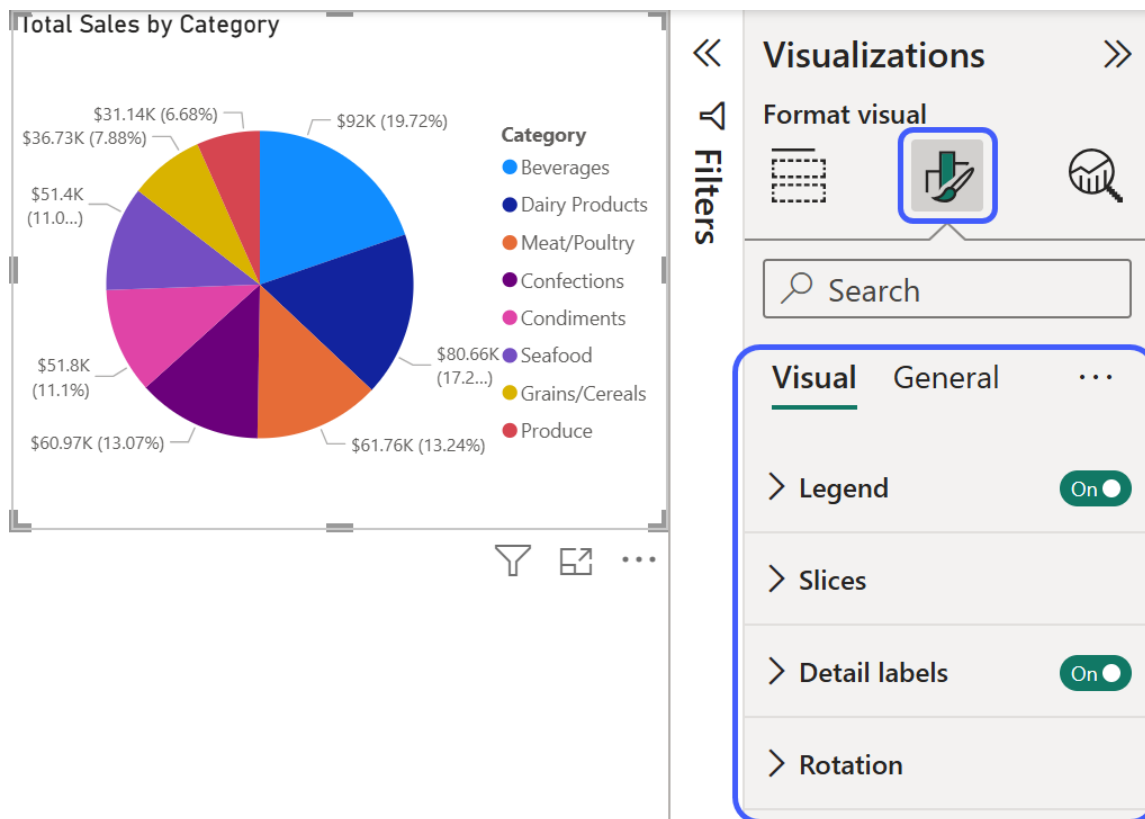
Working with Visualizations

Section 3.A

Customizing Visualizations and Report Interactions

3.A.24 Formatting Options

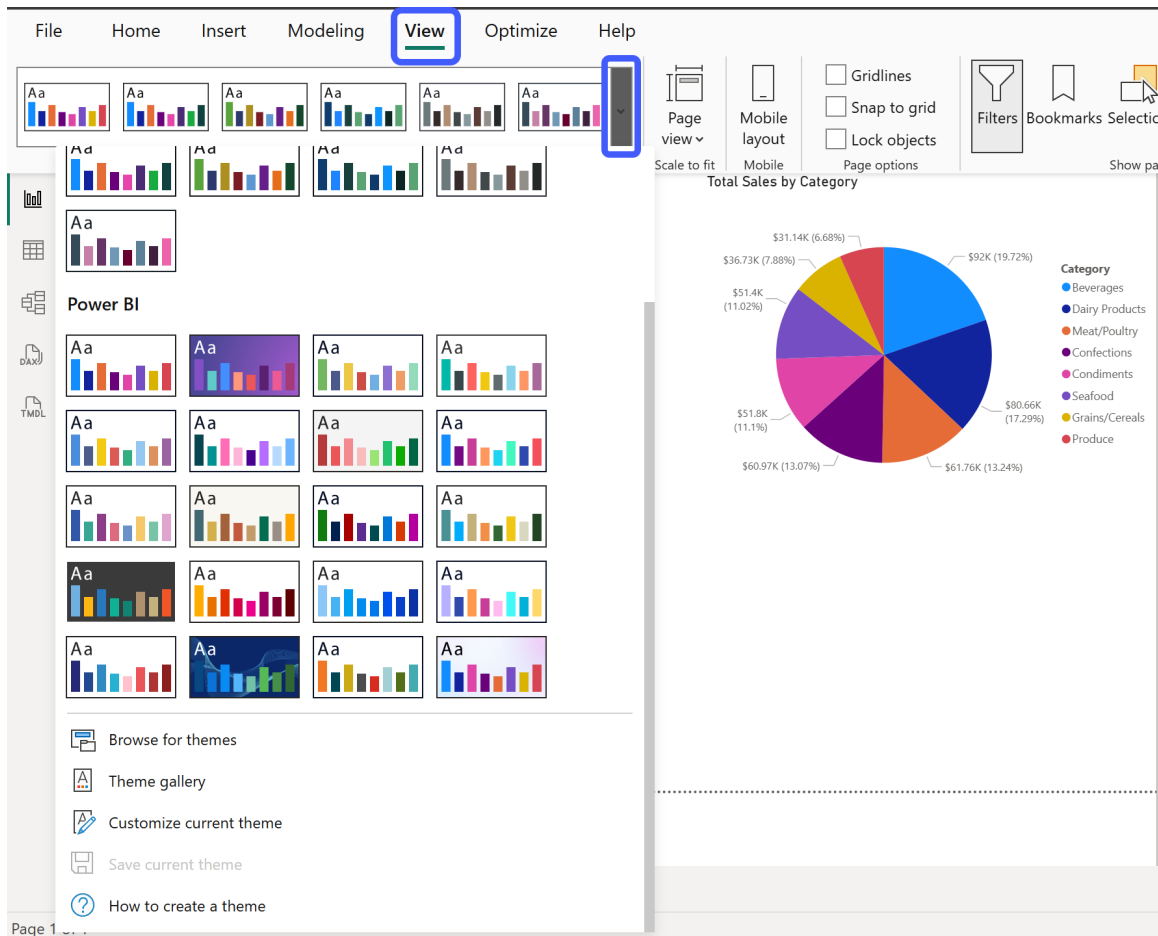
- Formatting can be applied at the level of each specific visualization
 - ◇ Formatting options include use of titles and text, colors, gridlines, borders, and other visual elements
 - ◇ When a visual is selected, these options can be accessed from the formatting tab of the Visualizations pane (icon: )



- The formatting options will vary depending on what visualization type is selected
 - ◇ Options specific to the visualized data appear under the Visual tab
 - ◇ More general formatting options appear under the General tab

3.A.25 Report Themes

- Themes apply a color scheme across a report
 - ◇ Available themes can be accessed from the View tab of the ribbon



- All report visuals conform to theme settings by default, unless other specific custom formatting is applied
- Custom themes can be created based on branding guidelines or other visual standards
 - ◇ Custom themes can be exported and imported, allowing users to reuse them across different reports and workspaces.

Section 3.B

Report Interactions

3.B.1 Filter Interactions

- By default, filter interactions apply across a report page as a user interacts with the report
- Clicking on an attribute of a visualization filters the report page to that attribute
- The Filters pane can be used to apply filters to a specific selected visual or across a report page

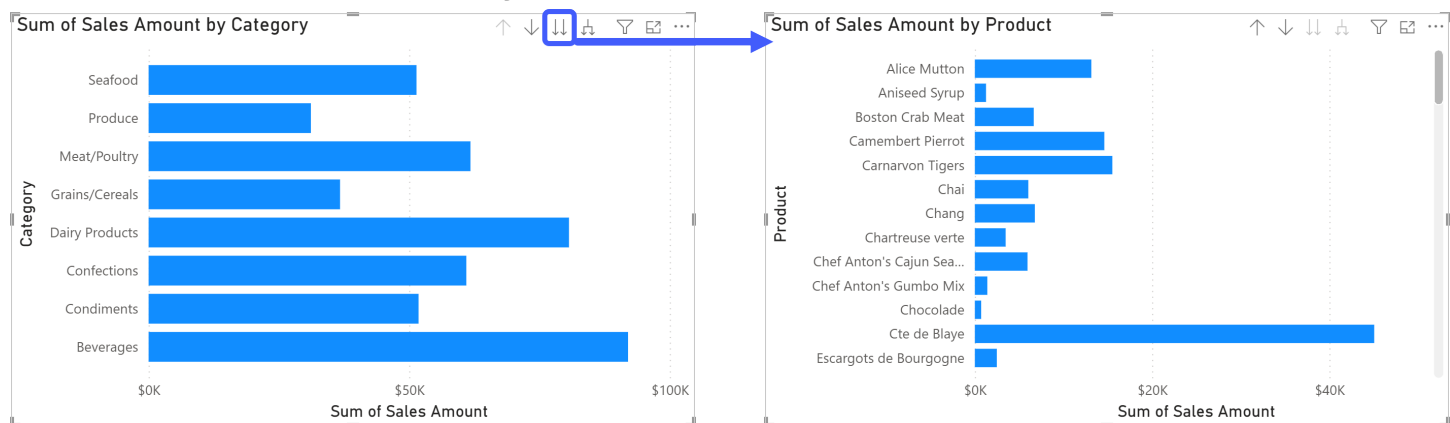
3.B.2 Move Down vs. Expand Hierarchy

- A hierarchy can either be expanded or drilled into, breaking down data into more granular levels

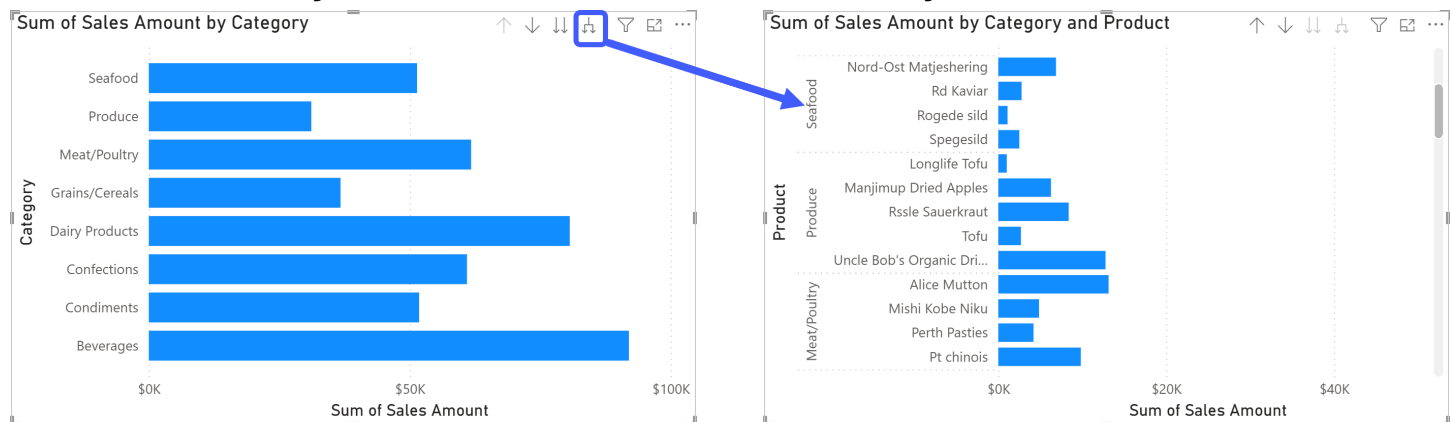
◇ Example: year > quarter > month

◇ Example: category > subcategory > sub-subcategory

- Go to next level double-arrow ↓↓** takes the visualized data down one level in the hierarchy

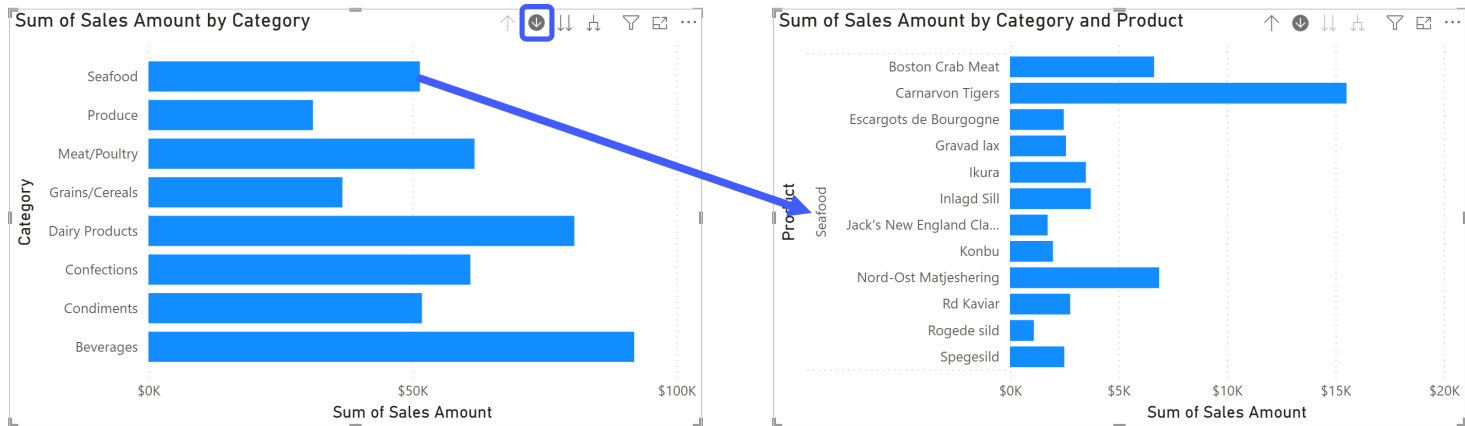


- Expand forked-arrow ↘** maintains the top-level categories but breaks down by the next level in the hierarchy

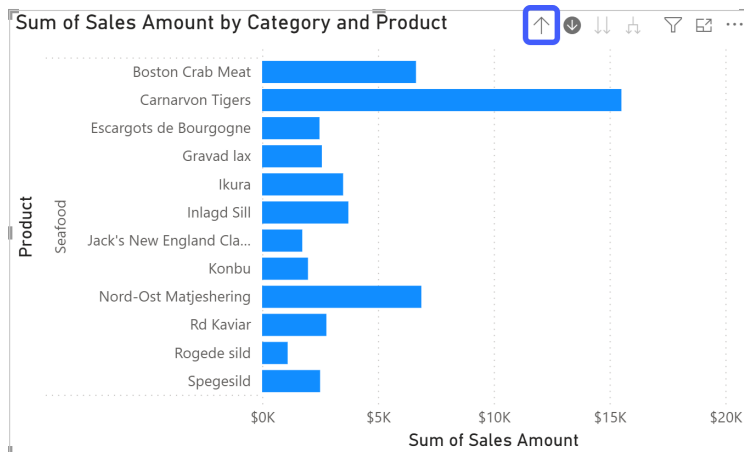


3.B.3 Drill Down, Navigate Up

- **Drill down** (single arrow, appears as ↓ when not on, ⬇ when on) lets you click on an element of the chart and filters down to the next level in the hierarchy for that element only



- To move up to a higher level in the hierarchy, use the up arrow ↑



- ◇ This works to navigate up no matter what type of hierarchy navigation was used to move or drill down levels

Section 3.C

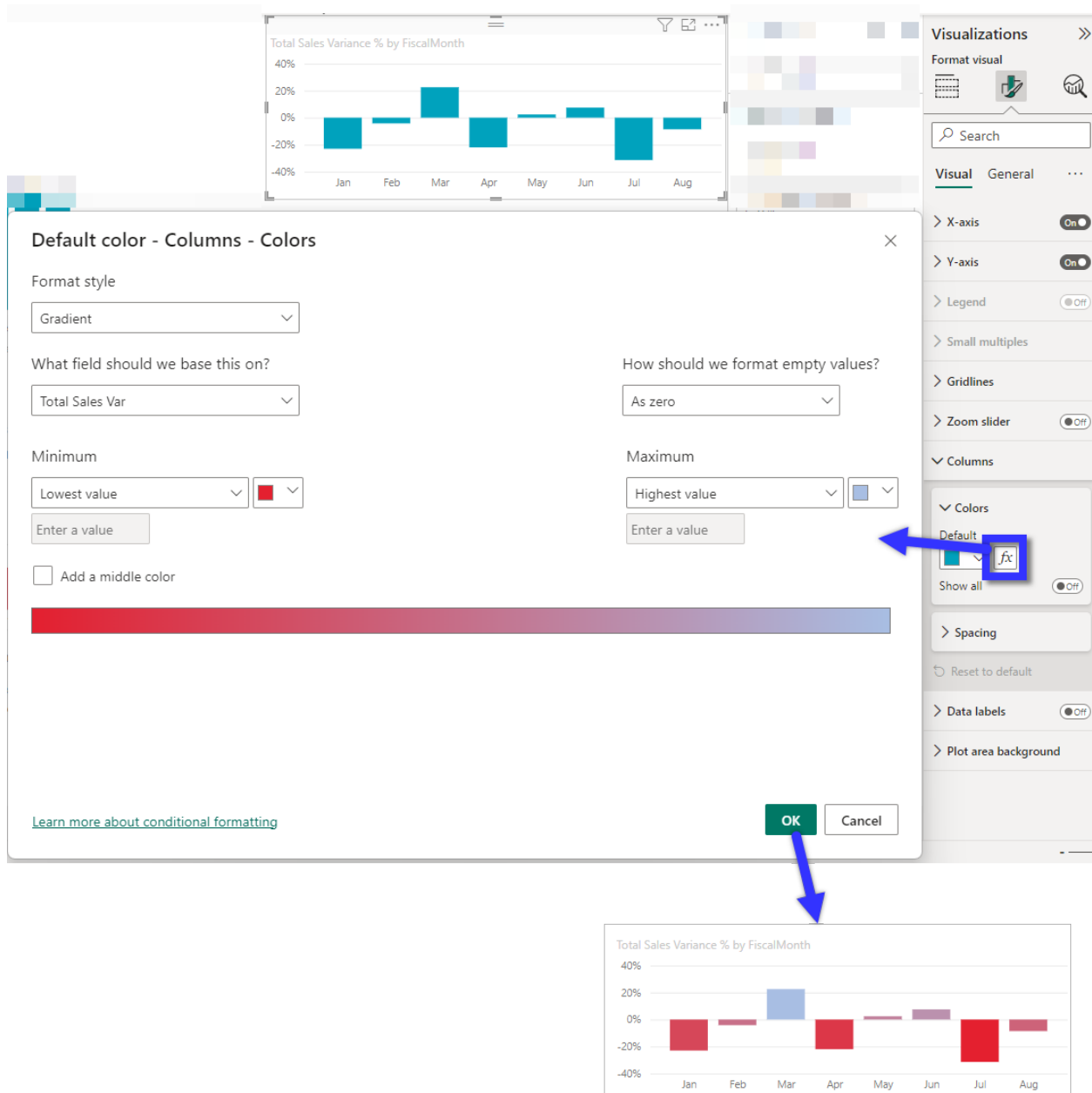
Data-Based Formatting

3.C.1 Customizing Visuals

- Visualizations can also be tailored to reflect attributes of the data in several ways
- Conditional formatting applies your specified formatting based on data values
- Grouping and binning put categories or values together
- Clustering is a machine-learning tool to highlight patterns and insights

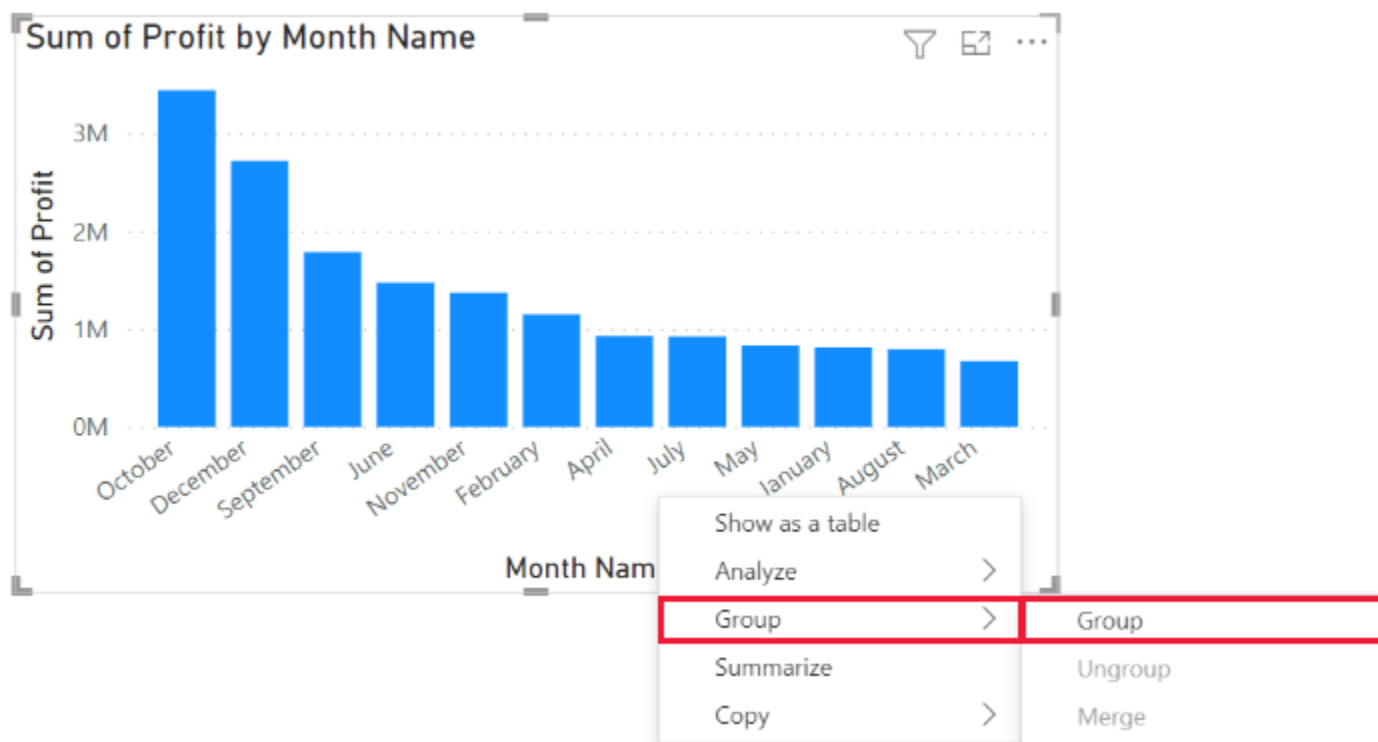
3.C.2 Conditional Formatting

- Allows users to visually highlight important data or trends in a report or dashboard
- Formatting based on data values, such as color-coding columns or adding data bars or icons to a table



3.C.3 Grouping

- Grouping allows users to combine data based on common attributes or characteristics



- Can help simplify complex data sets by reducing the number of categories or values displayed

3.C.4 Binning

- Binning allows users to set a bin size for numeric or time fields
- Number and size of bins can be customized, as well as the ranges and labels for each bin
- Created using “New group” and managed with the Groups dialog

Groups

Name * Open Month No (bins) Field Open Month No

Group type Bin Bin type Size of bins

Min value 1 Max value 12

Binning splits numeric or date/time data into equally sized groups. Enter bin size.

Bin size * 1

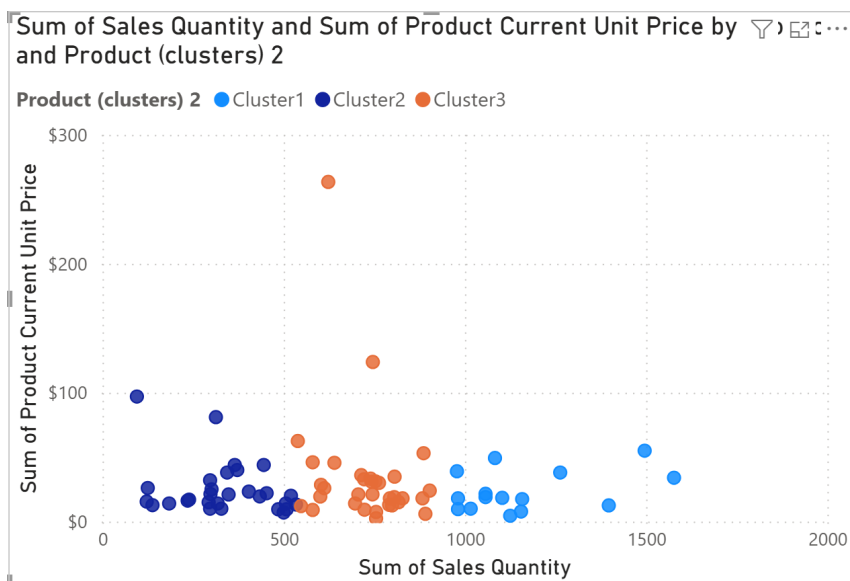
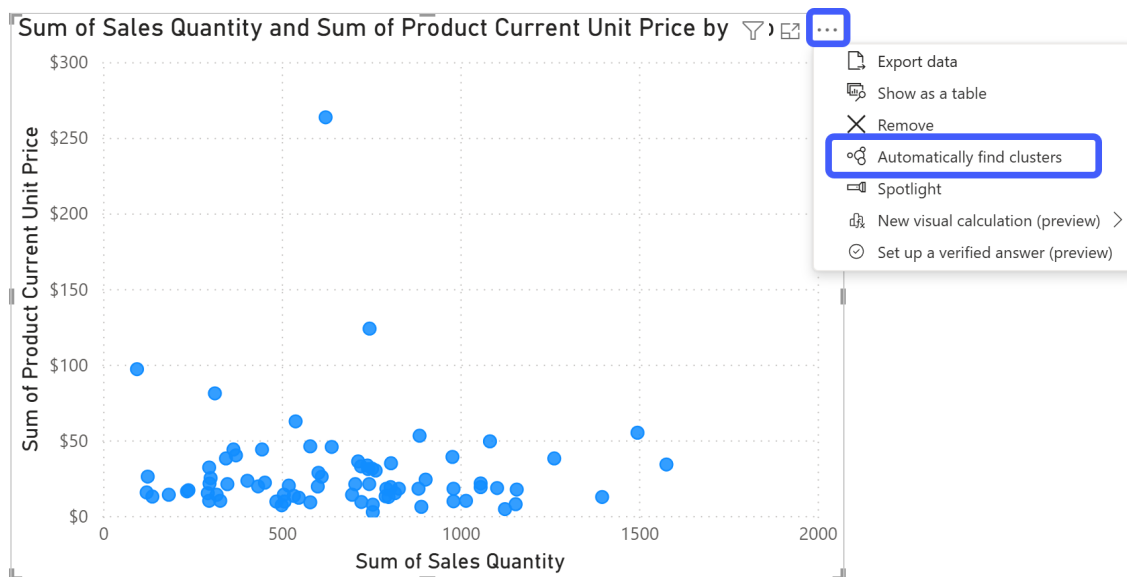
Reset to default

OK Cancel

- Note that you can make bins for calculated columns but not for measures (discussed in detail further in the course)

3.C.5 Clustering

- Available for scatter plot visualizations, **clustering** uses machine learning to group data points with similar characteristics
 - ◇ Can help identify patterns and insights in complex data sets that may not be immediately apparent
 - ◇ Found under the ... icon at the corner of a selected scatter plot

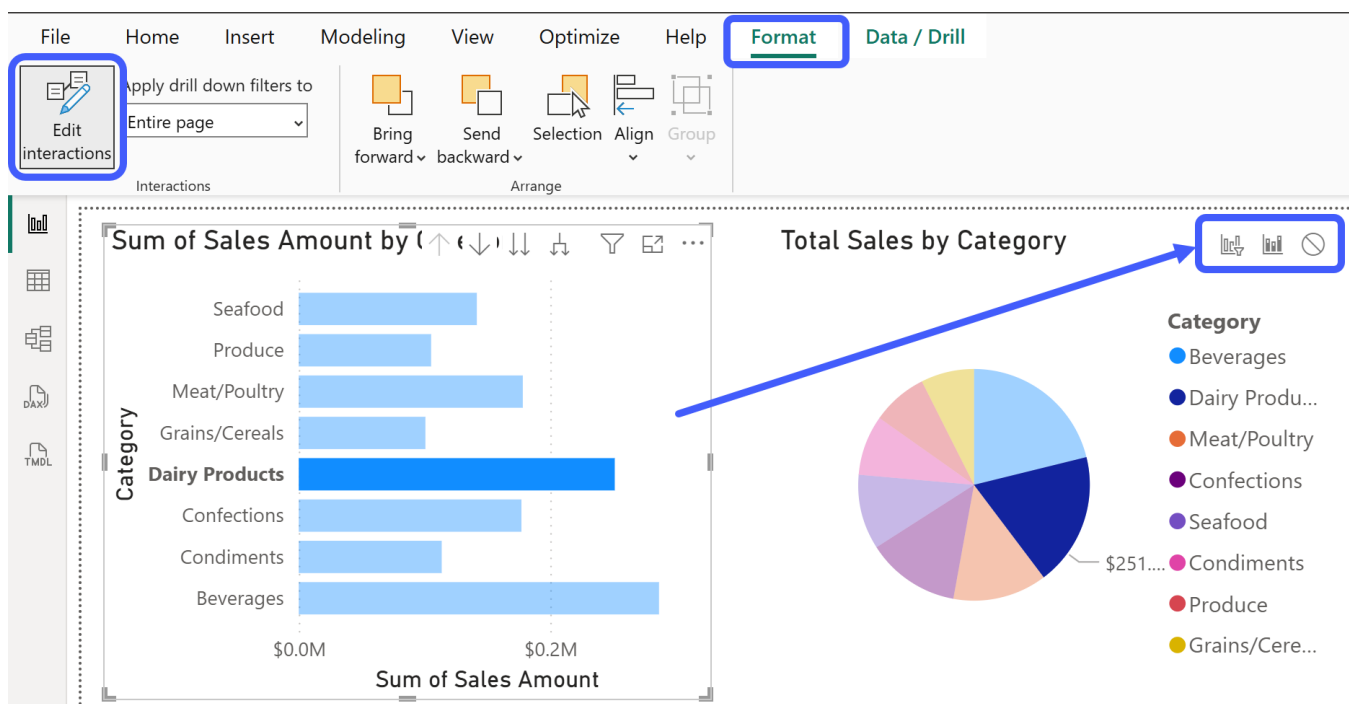


Section 3.D

Adding Interactive Features

3.D.1 Customizing Filter Interactions

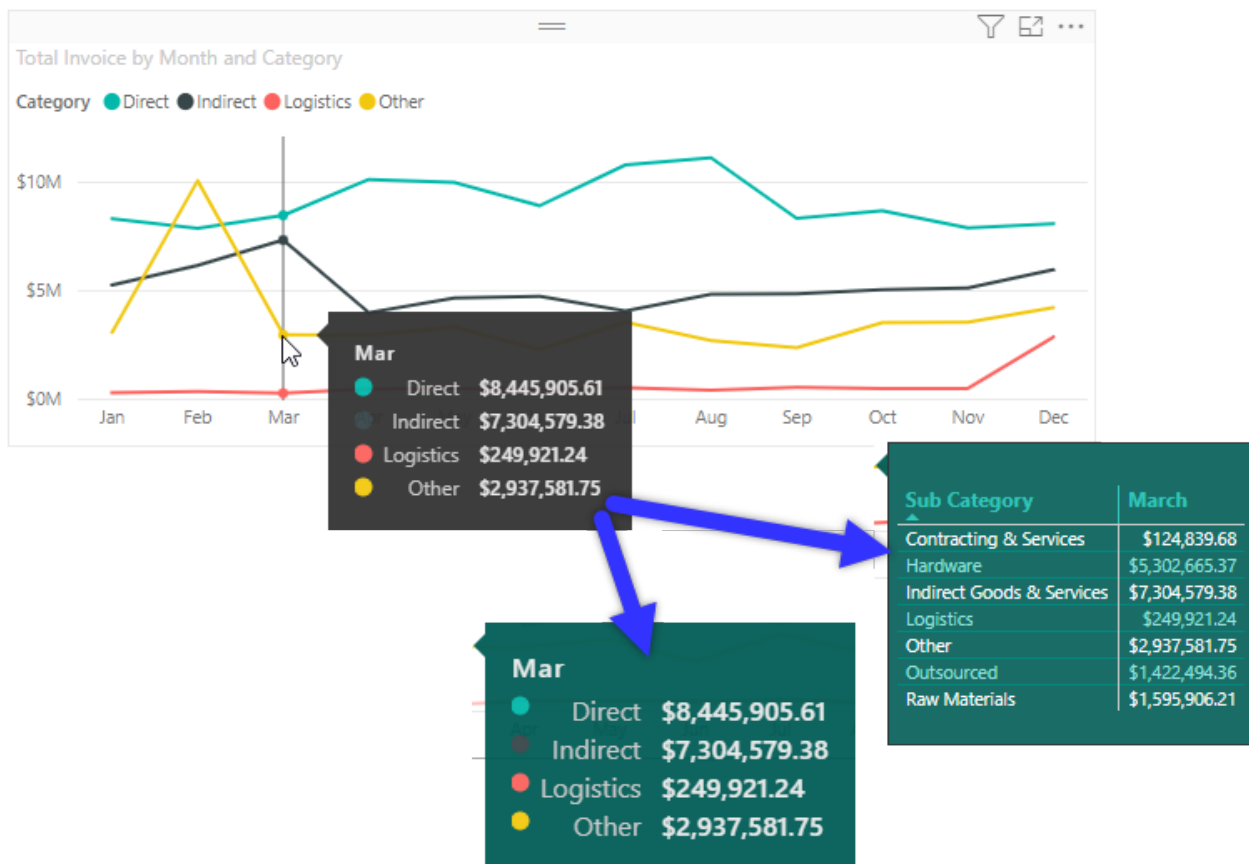
- Sometimes a filter applied to one visual can affect the other visuals in unintended or unhelpful ways
- Similarly, when multiple filters are applied in a Power BI report, they can sometimes compete with each other or become overly granular in combination
- Users can control how filters *applied to the selected visual* impact *other* visuals with the Edit Interactions function



- By editing filter interactions, you can prioritize specific filters for particular visuals, ensuring that the relevant data is highlighted correctly

3.D.2 Customizing Tooltips

- Tooltips can be customized in both *how* and *what* they display, from simply changing formatting to adding a custom tooltip page to the report
- Custom tooltips can be used to provide more detailed information about specific data points
- Tooltips can enhance interactivity, and can also be used to provide accessibility features, such as including audio or descriptive text

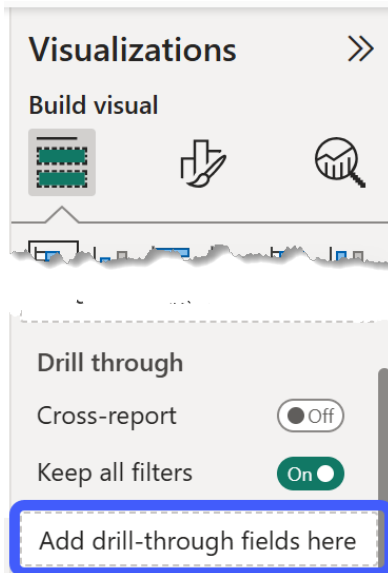


3.D.3 Drill Mode

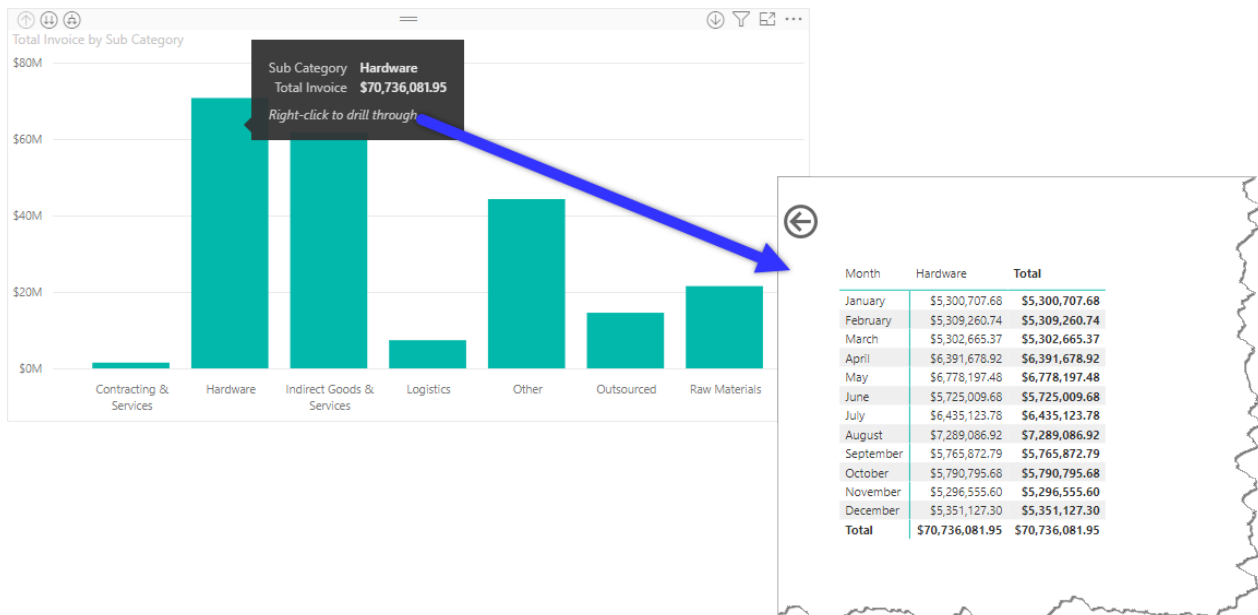
- Turning on Drill Mode for a visualization applies a drill-down interaction when clicking on a data point
- Drill Mode can be turned on using the down arrow ↓ in the corner of a visualization (changes to ⬇ when turned on)
- Once a drill-down filter is applied, user can drill up with the up arrow ↑ used for hierarchy navigation

3.D.4 Drill-through Pages

- Provide a means for report users to explore specific data points in more depth without cluttering the main report page
- ◇ Drill-through behavior is enabled by adding a field used elsewhere in the report to the drill-through well of another report page

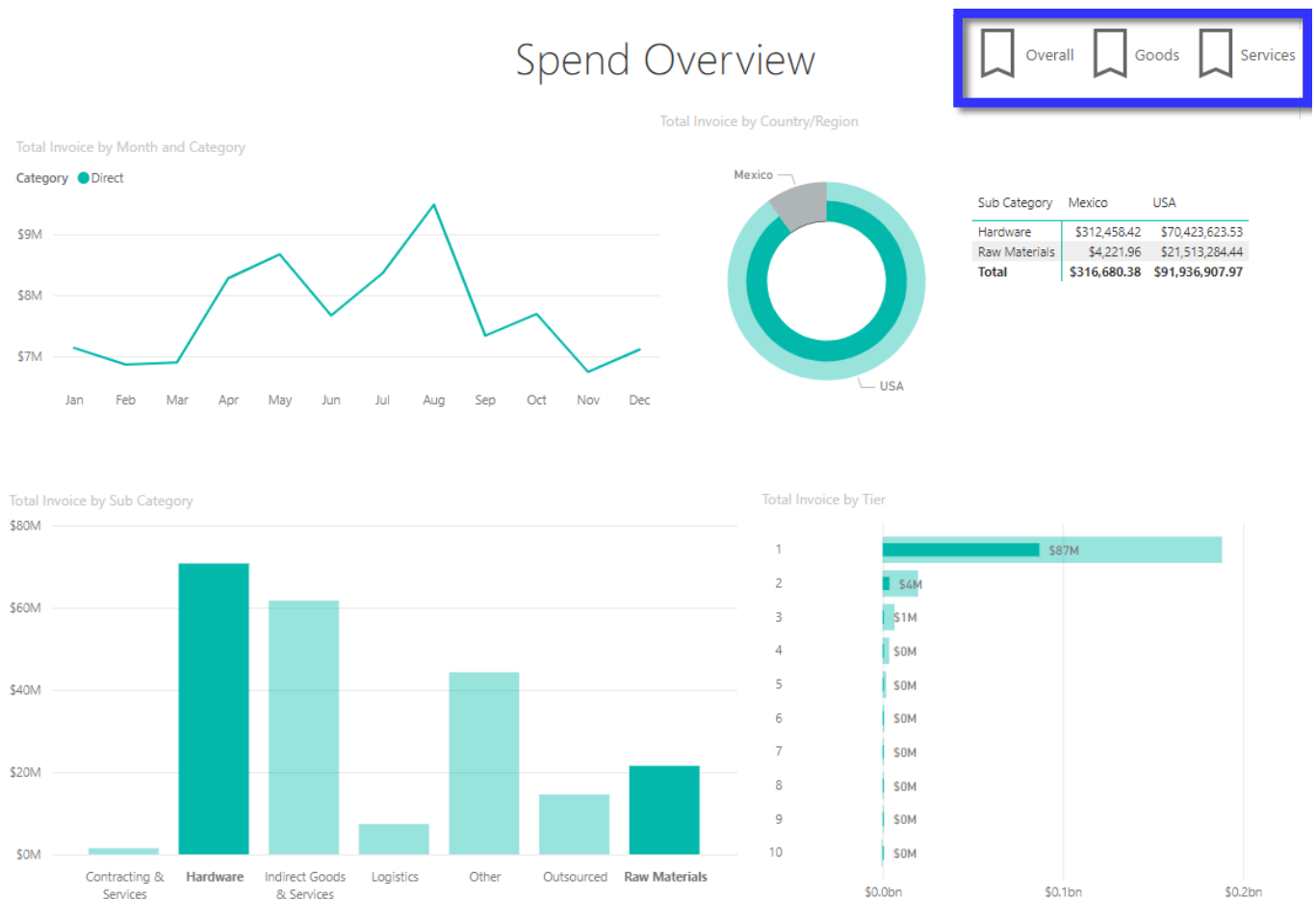


- ◇ Filters applied at the starting page will pass through to the drill-through page unless other behavior is specified



3.D.5 Bookmarks

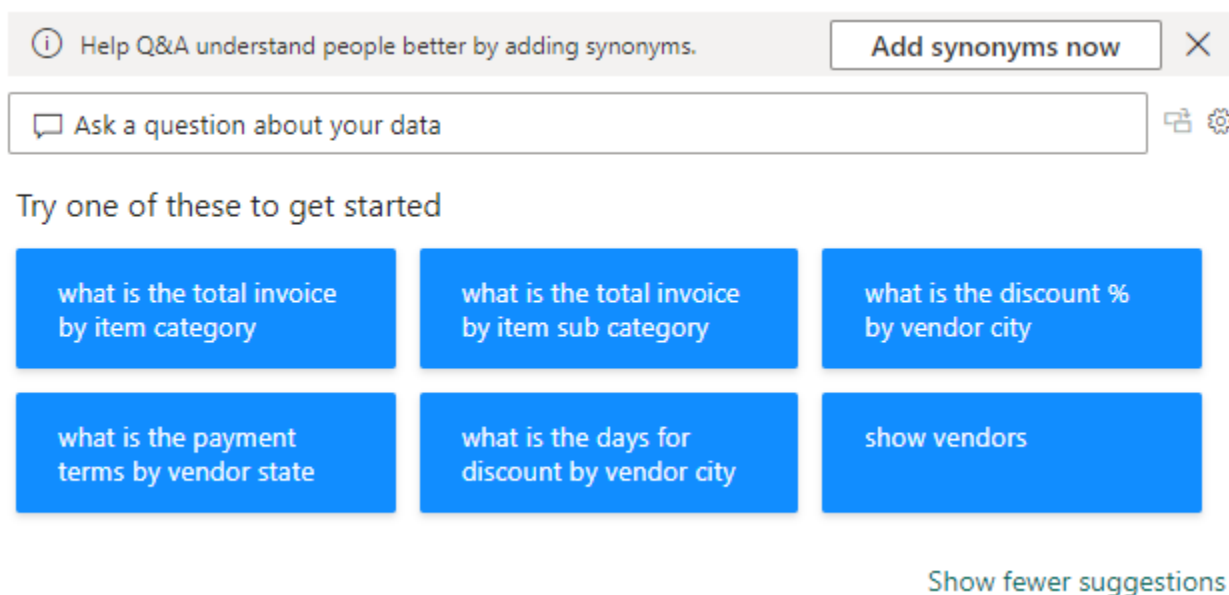
- Bookmarks allow you to capture and navigate to specific states of a report, including applied filters
 - ◇ Encourages interactivity by creating dynamic display of different scenarios or perspectives
 - ◇ Also can be used to provide convenient shortcuts to support key insights



- The control panel for Bookmarks can be turned on/off from the View tab of the ribbon
 - ◇ New bookmarks can be added and existing bookmarks managed from the Bookmarks panel

3.D.6 Q&A Visualization - Deprecated

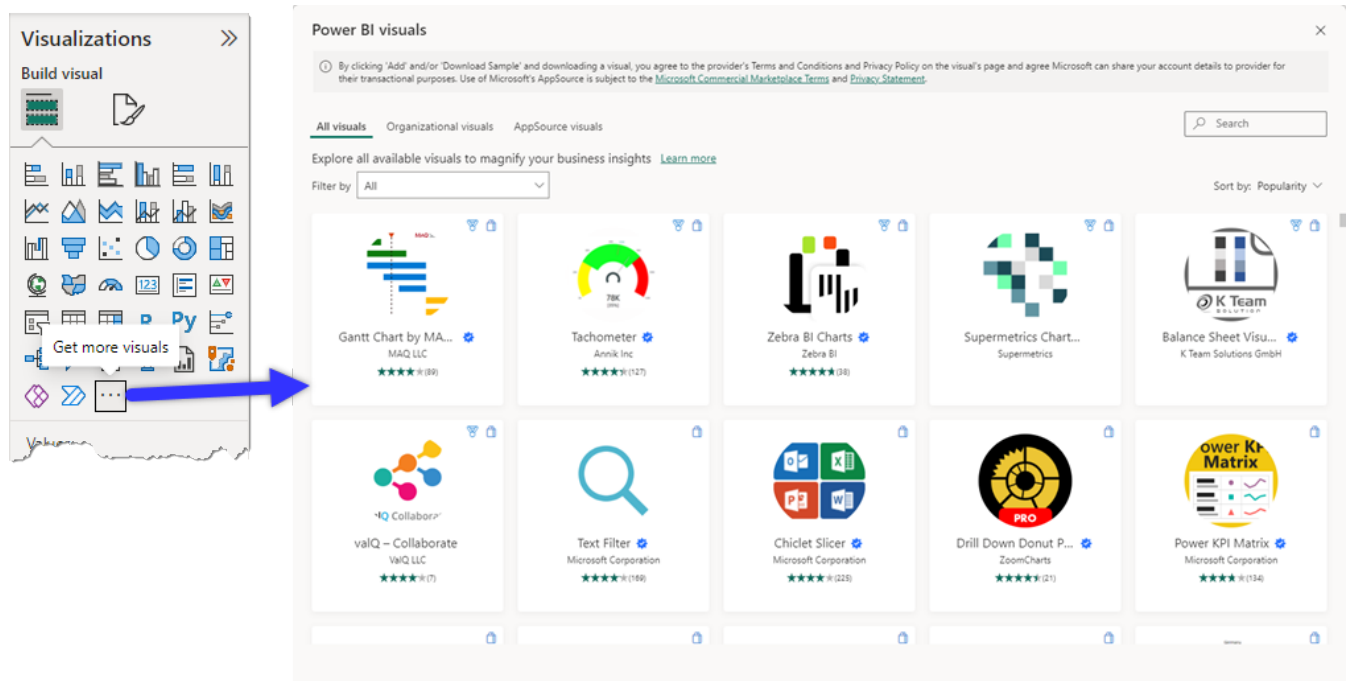
- The Q&A visualization has been available for several years as a way to enable users to ask questions about the report data using a simple plain language query
 - ◇ Because it uses natural language, Q&A can understand synonyms and variations of words, allowing report users to query the data in a more intuitive way
 - ◇ Synonyms can also be added, especially helpful if your data contains specialized vocabulary
- It can be used to generate a variety of visualizations, such as tables, charts, maps, and timelines.



- While you will still see this visualization appear in sample reports, it is being discontinued by Microsoft and will no longer be supported in the future

3.D.7 Custom Visualizations

- Custom visualization options are available via AppSource, from Microsoft and other providers
- Many useful and creative visualization types are available:
 - ◇ Chiclet slicer (using custom images)
 - ◇ Gantt, Sankey, sunburst, tornado, and other charts
 - ◇ Enhanced KPIs and gauges
 - ◇ Animated charts and slicers to add visual engagement



3.D.8 Week 10 Wrap-Up

- This week moved from foundations to fluency across all three layers of a Power BI solution: the transformation layer, the calculation layer, and the report layer
 - ◇ Module 1 — Power Query Formula Language (M) took you behind the GUI and into the code to read, modify, and write queries directly: working with `let/in` structure, dependency-based calculation, and lists, records, tables, and functions.
 - ◇ Module 2 — Working with DAX introduced the two evaluation contexts that govern every DAX calculation:
 - * Row context is the frame a calculated column works in (one row at a time), and functions like `RELATED()` and `RELATEDTABLE()` let you reach across relationships while staying in that frame
 - * Filter context is the frame a measure works in, shaped by slicers, report page filters, and wherever the measure is placed on a visual
 - ◇ Module 3 — Working with Visualizations introduced options for custom formatting and interactivity: conditional formatting, grouping, binning, and clustering let the data itself drive how the report looks, while drill-down, drill-through pages, bookmarks, and customized filter interactions give report users control over what they see
- Coming up in Week 11: The focus shifts from building reports to sharing them, using Power BI Desktop to publish to the Power BI Service, exploring dashboards, and getting reports to the people who need them

End of Week 10 Student Guide